Cairo University
Faculty of Computers and Artificial Intelligence
Artificial Intelligence Department
Information Systems Department

Seed Bank

Quality Intelligence Platform

Graduation Project Final Documentation

Prepared by:

Omar Ezz-ElDein Abdullah	20220228
Youssef Ahmed Awad	20220385
Ali Abdulrahman Mohamed	20220213
Youssef Tarek Ali	20220397
Mohamed Amr Mahmoud	20220302

Supervised by:

Dr. Eman Ahmed
Dr. Ali Zidane
Dr. Heba Sherif
Dr. Ghada Dahy

Academic Year: 2025/2026

Contents

List of Figures

List of Tables

List of Abbreviations

API	Application Programming Interface
CBAM	Convolutional Block Attention Module
CNN	Convolutional Neural Network
COCO	Common Objects in Context (dataset format)
CPU	Central Processing Unit
CSV	Comma-Separated Values
FCAI	Faculty of Computers and Artificial Intelligence
GMP	Global Max Pooling
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
mAP	mean Average Precision
ML	Machine Learning
R-CNN	Region-based Convolutional Neural Network
REST	Representational State Transfer
RTL	Right-To-Left
UI	User Interface
YOLO	You Only Look Once

Abstract

English Abstract

Seed quality decides how well a crop germinates and how much it finally yields, yet it is still judged by hand. An inspector pours out a sample, spreads it on a tray, and sorts the seeds into good and bad piles one at a time. That work is slow, it tires the eye, and two people often disagree on the same seed, so it can only ever check a tiny part of a large batch. Seed Bank is a prototype that does this job automatically. A user takes a photo of a batch of seeds; the system finds every seed in the picture, decides whether each one is good or bad, and reports how many seeds there are, what share of them are good, and a breakdown by crop. It currently handles two crops, coffee and maize. Teaching a computer to find the seeds needs many example photos in which every seed is already marked, and those are expensive to prepare by hand. A companion tool, MultiSeedGen, solves this by building such training pictures automatically, so the system can learn without anyone marking thousands of photos. On unseen test images the prototype finds nearly every seed and grades each one correctly about 97% of the time, and it does so on an ordinary computer. This shows that fast, consistent, automatic seed grading is within reach of the small laboratories and farmers who cannot afford industrial sorting machines.

Arabic Abstract

تُحدّد جودة البذور إلى حدٍّ كبير نسبة الإنبات والمحصول النهائي، ومع ذلك ما زال تقييمها يجري يدويّاً. يسكب المُصنِّف عيّنةً من البذور، وينشرها على صينية، ويفرزها بذرةً بذرة إلى سليمة وتالفة. هذا العمل بطيء ويُجهد العين، وكثيراً ما يختلف شخصان على البذرة نفسها، ولذلك لا يمكنه فحص سوى جزء ضئيل من الدُّفعة الكبيرة. «Seed Bank» نموذج أوّليّ يؤدّي هذه المهمّة تلقائيّاً؛ إذ يلتقط المستخدم صورةً لدُفعة من البذور، فيعثر النظام على كلّ بذرة في الصورة، ويقرّر ما إذا كانت سليمة أو تالفة، ثمّ يعرض عدد البذور ونسبة السليم منها وتوزيعاً بحسب نوع المحصول. ويتعامل النظام حاليّاً مع محصولين، هما البُنّ والذُّرة. ويحتاج تعليمُ الحاسوب العثورَ على البذور إلى عدد كبير من الصور التي وُسِمت فيها كلّ بذرة مسبقاً، وهي صور مكلفة الإعداد يدويّاً. تسدّ الأداة المرافقة «MultiSeedGen» هذه الحاجة ببناء صور التدريب هذه تلقائيّاً، فيتعلّم النظام دون أن يَسِم أحدٌ آلاف الصور. وعلى صور اختبار لم يَرَها النظام من قبل، يعثر النموذج الأوّليّ

على كلّ البذور تقريباً، ويصنّف كلّ بذرة تصنيفاً صحيحاً نحو ٪97 من الحالات، ويفعل ذلك على حاسوب عاديّ. وهذا يبيّن أنّ تقييم جودة البذور تقييماً سريعاً ومتّسقاً وتلقائيّاً صار في متناول المختبرات الصغيرة والمزارعين الذين لا يقدرون على شراء آلات الفرز الصناعية.

Chapter 1

Introduction

1.1 Background and Motivation

Seed quality decides a large part of what a farmer harvests. A sample that looks fine to the eye can still carry a high fraction of broken, immature, discoloured, or diseased grains, and that fraction sets the germination rate and the final yield. Before a lot is planted, traded, or priced, someone has to judge how good it is. In most of the supply chain that judgement is still made by a human grader who pours out a sample, spreads it on a tray, and sorts the seeds by hand into good and bad piles, counting as they go.

Manual inspection has two problems that are hard to fix by training people better. It is slow: a careful grader works through a few hundred seeds before fatigue sets in, which is a tiny sample of a multi-tonne lot. It is also subjective. Two graders looking at the same tray will disagree on the borderline seeds, and the same grader will drift over a long shift. The result is a quality number that is neither repeatable nor auditable, which is a weak basis for a price or a planting decision.

There is also a technology gap. Large operations use industrial optical sorting machines, but those are expensive and complex enough to be out of reach for smaller laboratories, independent researchers, and individual farmers. On the other side of the gap there are no digital tools at all, so smaller operations fall back on subjective estimation or inconsistent sampling. Nothing affordable sits in the middle, and a large part of the agricultural community has no access to precision quality control.

Automating the task is harder than it sounds, because seeds are not manufactured parts on an assembly line. They are organic and irregular, they vary in texture and colour, and in a real sample they touch and overlap. General-purpose detectors struggle in those dense clusters: two touching seeds get read as one, and subtle surface defects get missed. A grader that works on real trays has to handle that clutter rather than assume one clean seed per frame.

Computer vision still fits this task well once the clutter is taken seriously. The decision a grader makes, deciding whether one seed is good or bad, is a per-object visual classification,

and the wider job of finding every seed in a tray is object detection. Both have matured to the point where they run on commodity hardware and beat a tired human on consistency. Surveys of deep learning in agriculture report strong results across crop and produce inspection once enough labelled data is available [1], and the same convolutional models that classify plant disease from leaf photographs transfer to grading the seeds themselves [2]. An automated grader does not get tired, scores every seed in the photographed sample, and produces the same answer twice for the same image.

This project, *Seed Bank*, builds that automated grader as a working prototype rather than a notebook demo. It targets two crops, coffee and maize, which are visually distinct enough to exercise the two-crop routing in the pipeline. A user photographs a batch of seeds; the system locates each seed, scores its quality, and reports the aggregate good-rate, seed count, and a per-crop breakdown.

1.2 Problem Definition

Two linked problems sit at the centre of this project. The first is the shape of the machine-learning task itself. The second is a data problem that the first one exposes, and it is the harder of the two.

1.2.1 Reading a tray photo takes two steps, not one

A photo of a tray holds many seeds at once, so the computer cannot judge the whole picture in a single look. The work splits into two steps that run one after the other.

The first step is to find the seeds: look at the whole photo, mark where each seed is, and note which crop it belongs to, coffee or maize, since one tray can hold both. This turns a single photo into a set of located seeds.

The second step is to grade them: take each seed the first step found, look at it on its own, and decide whether it is good or bad. Coffee seeds are graded by the part of the system trained on coffee and maize seeds by the part trained on maize, because the two crops look nothing alike. One photo therefore becomes many seeds, and many seeds become many good-or-bad verdicts that add up to the final report.

The two steps are kept separate on purpose. Finding seeds and grading seeds are different jobs that learn from different examples, so keeping them apart lets either one be improved or replaced without disturbing the other. It also means they need different training data, which is where the second problem appears.

1.2.2 The dataset gap

The two steps do not need the same kind of data, and that mismatch is the problem this project had to solve before anything else could work.

The grading step is easy to train. It learns from pictures of one seed at a time, each labelled good or bad, and pictures like that are everywhere: a single seed on a plain background is cheap to photograph and quick to label. Coffee and maize both have plenty of them.

The finding step is the hard one. To learn where seeds are, it needs photos of many seeds together, with every seed in the photo already marked. That kind of data barely exists. Almost every public seed dataset shows one seed at a time, which teaches nothing about finding seeds in a crowd, where they touch and overlap. Making such a set by hand means photographing trays of scattered seeds and marking every grain in every photo, hundreds of marks per photo across thousands of photos. That marking effort is the bottleneck, and it is slow and costly enough to stop most teams before they start.

So the situation is lopsided. The data needed to train the grading step is plentiful, while the data needed to train the finding step is scarce and expensive to produce. Closing that gap is what motivated the second half of this project, *MultiSeedGen*: a tool that takes the single-seed pictures we already have and builds marked many-seed photos from them, so the finding step can be trained without hand-marking thousands of trays.

1.3 Project Objectives and Proposed Solution

The objectives follow directly from the two problems above:

- Build a complete grader that takes a photo of a seed batch and returns a grading report for coffee and maize.
- Keep a record of which trained model produced each result, so any verdict can be traced back later.
- Deliver the grader as a web app that a farmer can actually use.
- Provide basic model management and a way to score a model against a labelled dataset before it is used.
- Close the data gap by generating the marked many-seed training photos the finding step needs.

The solution has two parts that match the two groups of objectives.

Seed Bank is the platform and prototype. A user uploads photos through a web page; behind it the system finds the seeds, grades each one, stores the results together with a note of which trained model produced them, and sends back the report. A small amount of model management sits around this, enough to keep track of model versions and to check a model against a labelled dataset before it is put to use.

MultiSeedGen is the data generator. It cuts individual seeds out of source photos, drops many of them onto realistic backgrounds with varied lighting and a touch of camera-like noise,

and saves the result as a ready-to-train dataset with every seed already marked. Pasting real cut-outs onto varied backgrounds is a known and effective way to produce labelled training images cheaply [3], and varying the lighting, scale, rotation, and noise follows the domain-randomisation idea of pushing a model to generalise by showing it a wide spread of synthetic variation [4]. Because the tool places every seed itself, it knows exactly where each one is, so the labels come for free.

1.4 Project Scope and Limitations

In scope. The grader covers two crops, coffee and maize, and is built so a third crop is added by registering a new detector class and a matching classifier rather than by rewriting the pipeline. It is delivered as a working prototype with a web interface, and it supports offline evaluation of a trained model against a labelled dataset. MultiSeedGen covers the full path from single-seed crops to annotated multi-seed images.

Limitations. Seed Bank is a prototype rather than a commercial product; the goal was to show the pipeline works end to end, not to ship a hardened deployment. Capture assumes a top-down photo from a single camera, so the system has not been validated against angled shots or varied capture rigs. The largest open risk is the synthetic-to-real domain gap: a detector trained only on MultiSeedGen output learns the statistics of synthetic scenes, which never match real trays perfectly. Narrowing that gap with augmentation and realistic degradation is the tool's central design concern, but it cannot be assumed closed.

1.5 Development Methodology

The work followed an iterative, phased approach. Rather than building everything at once, each phase delivered a usable slice and was exercised before the next was stacked on it: first the problem framing and the synthetic-data generator, then the grading pipeline, then the web prototype around it. The detector could not be trained until synthetic detection data existed, so MultiSeedGen came early and the platform work followed it.

The machine-learning side ran as a tighter experiment loop. The first step was curating and cleaning the datasets, since raw public corpora carry mislabelled and duplicated images that would otherwise leak between splits. From there the team trained a sequence of model variants and measured each one on validation data and on a held-out test set before deciding what to change next. The classifier went through four versions: V1 was the baseline backbone, V2 added a CBAM attention module [5], V3 added hybrid pooling, and V4 changed a stride in the backbone to keep more spatial detail. Each change was kept only if it improved the held-out numbers, and the full set of results is reported in the evaluation in Chapter 6.

1.6 Tools and Technologies Used (Software and Hardware)

The stack is conventional where it can be, so that each choice is a well-understood tool with a known failure mode. The backend is built on Python and FastAPI [6]; the models are built and trained with PyTorch and torchvision, with Ultralytics YOLO and OpenCV alongside; relational data is stored in PostgreSQL, while analytics and telemetry data are managed in ClickHouse; the web client is React [7]; and the whole system runs under Docker [8]. Table 1.1 groups the set by area.

Table 1.1: Tools and technologies used, grouped by area.

Area	Tools
Backend	Python, FastAPI.
ML	PyTorch, torchvision, Ultralytics YOLO, OpenCV.
Data	PostgreSQL, ClickHouse.
Frontend	React.
Tooling	Docker.

On the hardware side, the models were trained on a workstation with an NVIDIA RTX 3060 GPU.

1.7 Project Timeline / Gantt Chart

The work was scheduled across the 2025/2026 academic year and ran in the phased order described in the methodology section. The early part of the year went to problem framing and to MultiSeedGen, because the detector could not be trained until synthetic detection data existed. The middle of the year built the grading pipeline and ran the model experiments, and the final stretch went to the web prototype and this report. Figure 1.1 shows the schedule.

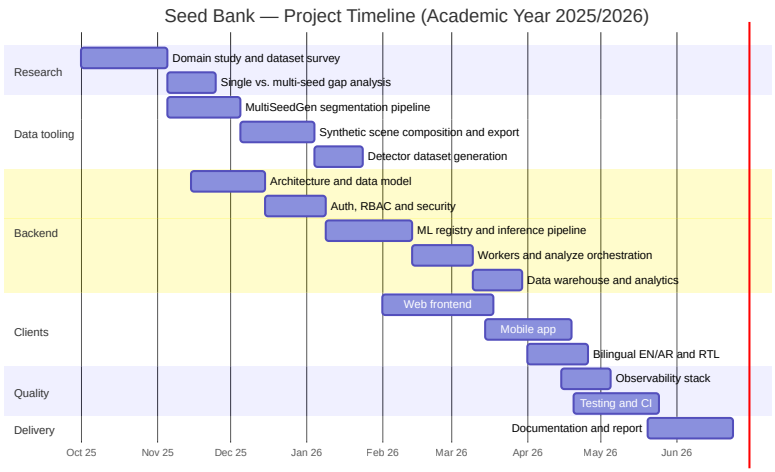

Figure 1.1: Project timeline (Gantt chart).

1.8 Report Organization

The rest of this report is organised as follows.

Chapter 2, Related Work, reviews the background this project builds on: object detectors such as Faster R-CNN and YOLO and the feature-pyramid and attention ideas behind them, image classifiers from the ResNet family, the synthetic-data and augmentation literature that justifies MultiSeedGen, and prior applications of computer vision in agriculture.

Chapter 3, System Analysis, sets out the requirements. It defines the user roles, the functional and non-functional requirements, and the use cases the grader has to support, and frames them against the two problems from this chapter.

Chapter 4, System Design, describes the architecture: the grading pipeline, the data model and how a result is tied to a model version, and the web client.

Chapter 5, Implementation, covers how the design was actually built, the model builders and inference path, the orchestration of the detect-then-classify flow, and MultiSeedGen's generation pipeline.

Chapter 6, Experimental Results and Evaluation, reports the experiments: the datasets and how they were prepared, the detection and classification metrics across the model variants, and the latency of the end-to-end pipeline.

Chapter 7, Conclusion and Future Work, summarises what was delivered against the objectives and lays out the next steps, with narrowing the synthetic-to-real domain gap as the main one.

Chapter 2

Related Work

This chapter sets out the background the project builds on. It starts with how seed quality is checked today, by hand, in a working seed bank. It then surveys five existing systems that automate part of that job, and notes where each one falls short of what we need. From there it moves to the computer-vision ideas behind our pipeline (object detectors and attention classifiers), the synthetic-data techniques that let us train a detector without labelling thousands of seeds by hand, and a short review of where AI sits in agriculture more broadly. The chapter closes with a comparison of prior work against what Seed Bank delivers.

2.1 Traditional methods of seed analysis

Before a seed batch is accepted into a seed bank for long-term storage, it has to pass a physical inspection that decides whether the seeds are fit to keep. For most of the history of seed banking this has been done entirely by hand, relying on the eye and experience of the person doing the grading.

The standard procedure is a physical examination carried out by a trained taxonomist or seed-bank technician. It is often called a purity analysis. The technician looks over the seed sample under a magnifying lens, a stereomicroscope, or a light table so that small features are easier to see. The first job is to separate pure seed from inert matter such as soil, chaff, or plant debris, so that what gets stored is actually seed and not background material taking up freezer space.

Once the pure seed is isolated, the technician judges each one against a set of physical criteria: size, shape, surface texture, and colour consistency. Seeds that look shrivelled, cracked, or discoloured are set aside, as are seeds that show signs of insect predation or fungal infection. Only seeds that meet the visual standard for health and structural integrity are passed on to the next stage. This step matters because a single infected or damaged seed can spoil a whole storage container, so the cost of missing a bad seed is high.

The weakness of this process is that it is slow, it does not scale, and it varies between graders. Two technicians can disagree on a borderline seed, and the same technician can drift over a long

shift. That inconsistency, together with the sheer time it takes, is what motivates an automated alternative.

2.2 Existing solutions

To understand the current landscape we looked at five systems, ranging from industrial machinery to open-source research code. Each automates some part of seed inspection, but most of them target growth potential (germination) or species identification rather than the physical quality checks that long-term storage actually requires.

2.2.1 LemnaTec SeedAIxpert

The LemnaTec SeedAIxpert [9] is the reference point for large corporate seed labs and plant breeders. It is an all-in-one hardware and software product built around an imaging cabinet with controlled lighting, so that every photograph is taken under identical conditions and the AI software can produce reproducible scores. It is designed for high throughput, processing large quantities of seed and replacing subjective human grading with standardised numbers.

Relevance to our project: the strength of the system is also its barrier, which is accessibility. It depends on custom, costly hardware and careful calibration that is impractical for a typical university seed bank or a smaller conservation effort. Its closed design also means we cannot adapt its algorithms to the specific physical defects we care about. Our aim is to reach useful accuracy from ordinary photographs without a dedicated imaging rig.

2.2.2 PCS Agri Track

PCS Agri Track [10] is a commercial product aimed at nurseries, built to modernise inventory management and germination tracking. Unlike the industrial lab equipment it runs on mobile and web, which lowers the barrier to entry. It moves nurseries away from manual counting by digitising the tracking process and recording data over time, such as germination curves.

Relevance to our project: the main drawback is its reliance on cloud processing and a stable internet connection, which is not always available in the field. Users have also reported that the interface is hard to follow and that the models generalise poorly, working for common nursery seeds but struggling with the diverse or rare species a seed bank holds. It still needs manual intervention for edge cases, which undercuts the goal of automation.

2.2.3 Seedy

Seedy [11] is a mobile app for hobbyist gardeners, botanists, and educators. Its purpose is on-demand seed identification and personal collection management. A user photographs a seed

with a phone camera, and the app's recognition model returns the species along with metadata and growing guides.

Relevance to our project: Seedy shows that mobile seed analysis is both viable and easy to use, which supports our own mobile capture flow. Its scope, though, is strictly identification. It tells the user what a seed is, not how healthy it is. It does not measure physical metrics or scan for damage and disease, which is exactly the gap we are trying to fill.

2.2.4 GerminationPrediction

In the academic sphere the GerminationPrediction project of Grimm et al. [12] is a useful reference. It is open source, uses a Faster R-CNN detector trained on a public dataset of over twenty-three thousand images, and automates germination assessment by counting how many seeds in a batch have sprouted.

Relevance to our project: this is the closest match technically, since it also uses a CNN-based detector. The biological focus is different, though. They measure viability (whether a seed will grow), while we measure physical integrity (whether a seed is sound enough to store). A seed can be viable enough to sprout and still carry cracks or fungal spores that make it risky to store in a shared freezer. We borrow their open-source, reproducible approach but retrain the focus toward defect detection rather than sprout counting.

2.2.5 Multi-View Oil Palm system

The Multi-View Oil Palm project [13] pushes computer vision past the limits of a single image. It uses Multi-View CNNs (MVCNN) to analyse irregularly shaped oil palm seeds from several angles, which gives it a 3D understanding and strong accuracy on that crop.

Relevance to our project: this work makes a point we take seriously, that a single top-down photograph can miss defects hidden on the side or underside of a seed. Their multi-view setup gives richer data, but it is specialised for oil palm and needs a complex rig to capture multiple angles. We aim for a middle ground: high accuracy from accessible single-view photographs, without the computational and hardware cost of a multi-view system.

2.3 Computer-vision background

The components below are the model families our pipeline is built from. Seed quality assessment of a photograph that contains many seeds is really two problems, not one. First find each seed, then judge it. The first stage is object detection and the second is per-seed classification. We describe the ideas behind each here and leave the wiring and results to later chapters.

2.3.1 Two-stage detectors

Two-stage detectors first propose regions that probably contain an object, then classify and refine each proposal. Faster R-CNN [14] is the reference design. A region proposal network shares features with the detection head and generates candidate boxes cheaply, which removed the slow external proposal step that earlier R-CNN variants depended on. Sharing features lets the whole detector train end to end and run fast enough to be practical.

A backbone produces strong features at a single scale, but seeds in a photograph vary in size with camera distance and how the kernels are arranged. The Feature Pyramid Network [15] addresses this by combining the coarse, semantically strong features from deep layers with the fine, high-resolution features from shallow layers, giving the detector a consistent representation across scales at little extra cost. This combination is a sensible default when detection accuracy matters more than raw speed, which fits a grading workload where each photo is analysed once and the result is stored.

2.3.2 One-stage detectors

One-stage detectors skip the proposal step and predict boxes and class scores directly in a single pass. YOLO [16] introduced the idea by framing detection as one regression over a grid, trading some localisation accuracy for a large speed gain. Later versions of the family closed much of that accuracy gap, and YOLOv8 [17] is a widely used current release with good tooling for training and export. Seed Bank does not commit to a single family. The inference layer can run either a fast one-stage detector where latency matters, for example a conveyor-belt scanning station, or the more accurate two-stage detector elsewhere, without changing the calling code.

2.3.3 Backbones and benchmarks

Both families rest on a backbone network and are measured against shared public benchmarks. Residual networks made very deep classifiers trainable by adding identity skip connections, so gradients reach early layers without vanishing [18]. ResNet is the standard backbone choice, and its shallow member, ResNet-18, is small enough to run many seed crops per image quickly while still benefiting from pretraining. ImageNet [19] provides the large classification dataset that backbones are pretrained on, and COCO [20] provides the detection benchmark with its standard mean average precision protocol. These datasets matter to us in two concrete ways: the backbones we use are ImageNet-pretrained, and the synthetic data described in Section 2.4 is written in COCO format so generated datasets drop straight into standard detection training code.

Once a project trains more than one model it also needs a way to track what was trained, with which data, and how each version scored. Seed Bank keeps this record itself: each evaluation run stores its metrics in PostgreSQL alongside a Markdown report in object storage, so trained

models can be recorded and compared without standing up a separate experiment-tracking service.

2.3.4 Attention in the classifier

The second stage takes each detected seed, crops it from the original image, and decides whether that single seed is good or bad. This is binary classification on a small crop, and our classifier is a ResNet-18 backbone with a Convolutional Block Attention Module [5].

Attention refines what a fixed backbone pays attention to. CBAM is a lightweight module that learns two attention maps in sequence: a channel map that reweights which feature channels matter, and a spatial map that reweights which positions matter. It plugs into an existing backbone with almost no extra parameters and improved accuracy consistently in the original study. For a seed crop the attention block helps the classifier focus on the part that carries the verdict, such as a cracked or discoloured region, rather than spreading its response over the whole box. The pooled features then feed a small head that produces a good/bad score. We train a separate classifier per crop type, which is why the detector groups its boxes by seed type before classification runs.

2.4 Synthetic data and cut-and-paste augmentation

This is the area that decided whether the project was feasible. Training the detector needs images of mixed seeds with a bounding box around every kernel. Producing that by hand is the labelling bottleneck: an annotator has to click out hundreds of small, similar, overlapping objects per image, and a useful training set runs to thousands of images. The companion tool MultiSeedGen removes that step by generating labelled detection data instead of collecting it, and its design draws on the synthetic-data literature.

The core technique is cut-and-paste synthesis. Dwibedi et al. [3] showed that pasting cropped object instances onto varied backgrounds produces training data good enough for competitive detectors, because a detector mostly needs to see an object's local appearance in many contexts, not a globally coherent scene. The paste does not have to look realistic to a human. As long as the local patches are varied, the detector still generalises to real images. The payoff is that the label comes for free: when the generator places a seed cut-out at a known position and scale, it already knows the exact bounding box, so every synthetic image is perfectly annotated with no human in the loop.

Cut-and-paste only works if the object crops are clean, with the background removed down to the seed's true outline. For the easy cases, classical thresholding handles high-contrast backgrounds with no extra dependencies. For feathered edges the U^2-Net salient-object model [21] separates foreground from background, and for the hardest cases Segment Anything [22] produces high-quality masks from a box or point prompt. The learned backends are optional, so a

basic install stays light, and low-quality masks are dropped before they reach the compositing step.

A detector trained on synthetic images still has to transfer to real photographs. The standard answer is domain randomization: vary the synthetic data so widely that the real distribution looks like just another sample. Tobin et al. [4] introduced the idea for simulation, randomising textures, lighting, and placement so a model trained only on synthetic data transfers to the real world. MultiSeedGen applies the same idea by randomising backgrounds, per-seed geometry (scale, rotation, flip, shear, perspective), and photometric conditions (gamma, blur, noise, vignetting) so the generated set spans far more variation than a hand-collected set would. These transforms sit inside the broader augmentation literature surveyed by Shorten and Khoshgoftaar [23], which finds that geometric and photometric perturbation reduces overfitting across vision tasks, and libraries such as Albumentations [24] made these transforms fast and annotation-aware, which matters because moving an image must move its boxes with it.

One design point is worth calling out because it is easy to get wrong. The generator enforces a deterministic train/validation split in which a source seed reserved for validation never appears in any training image. This is the leakage-safe split. Without it the model in effect sees the validation seeds during training and reports an inflated score, so the split has to be decided at the source-seed level rather than the image level.

2.5 AI in agriculture

AI is now a routine part of modern agriculture rather than an experiment. The shift is driven by pressure on the sector: arable land is shrinking and climate conditions are more volatile, so the older ”yield at all costs” mindset has given way to a precision-first one, where efficiency and resilience matter as much as raw output. The survey by Kamilaris and Prenafeta-Boldú [1] documents this move across agriculture, from yield prediction to weed and crop classification to quality grading, all shifting from hand-crafted features to learned representations with steady accuracy gains where labelled data was available.

In precision farming, models combine satellite imagery, soil-sensor readings, and local weather forecasts into field-level prescriptions for how much water or input each area needs. Closer to our problem is fine-grained computer vision for quality control. Where ordinary detection just identifies a crop type, fine-grained methods pick up subtle within-class differences such as hairline fractures, surface discoloration, or early-stage fungal damage that often escape the human eye. Mohanty et al. [2] show both the promise and the catch: their CNNs reached high accuracy at recognising crop diseases from leaf photographs on a curated dataset, but accuracy dropped sharply on images taken under conditions different from the training set.

That last result points to the barriers the field still faces. The first is capital cost: sensors and autonomous machinery carry a high upfront price that keeps smaller operations out. The second is the digital divide, a widening gap between fully integrated commercial farms and

smallholders. The third is the one that shaped our own work most: dataset variability and domain generalization. Seed diversity and lighting changes break models trained on a narrow set, and the usual fix is to collect local data and retrain for the specific harvest. The gap between a clean benchmark and field photographs is the central practical problem for any agricultural vision system, and it is the reason we treat data variety as a first-class concern rather than an afterthought.

2.6 Comparative analysis

The components above are individually well established. What distinguishes this project is not a new model but how the pieces are assembled into one workable pipeline, and the decision to remove the data-labelling bottleneck with a purpose-built generator rather than working around it.

Most prior work in seed and grain vision is a single artifact: one classifier or one detector, trained against a hand-labelled dataset and evaluated once. Such a system answers one question, either ”where are the seeds” or ”is this seed good”, but not the combined question the domain actually asks, which is how many seeds are in this photo and how many of them are bad. It is usually trained on a hand-labelled dataset, which caps the dataset size and therefore the accuracy.

Seed Bank differs on each point. It chains a detector and a per-type classifier into one detect-then-classify pipeline, so a single photograph yields a located, per-seed good/bad verdict and an aggregate report. It supports multiple crop types, each with its own classifier, with detections grouped by type before the classification stage. And the labelling bottleneck is attacked at the source by MultiSeedGen, which generates perfectly labelled multi-seed data with controlled variety, so the detector can be trained at a scale that hand-labelling could not reach. Table 2.1 summarises the contrast.

Table 2.1: Typical prior work in seed and grain vision versus the Seed Bank platform.

Dimension	**Typical prior work**	**Seed Bank**
Scope	Single model: a detector *or* a classifier	Two-stage detect-then-classify pipeline producing per-seed verdicts and an aggregate report
Quality question answered	“where are the seeds” *or* “is this seed good”	“how many seeds, and how many are bad” over a mixed photo
Crop types	Usually one species, fixed at training	Multiple crop types, each with its own classifier; detections grouped by type
Training data	Hand-labelled, which caps dataset size	Synthetic cut-and-paste generation with free, exact labels and controlled variety

Dimension	Typical prior work	Seed Bank
Data integrity	Leakage and label errors are easy to miss	Leakage-safe split at the source-seed level, with controlled overlap and box clipping

The wider point is that the contribution sits at the system level. Each model family here is taken from published work and used as intended. The originality is in binding detection, per-type classification, and a synthetic-data generator into one tool that a non-researcher can run, and in attacking the labelling bottleneck head-on rather than living with a small hand-labelled dataset. The chapters that follow describe how that system is built and how it performs; the experimental numbers are reported in the evaluation in Chapter 6.

Chapter 3

System Analysis

This chapter turns the product idea from the introduction into a clear statement of what Seed Bank has to do and the limits it works within. It starts with the people who use the system and what each of them wants, then lists the functional and non-functional requirements that the design is measured against. The requirements map onto capabilities that exist in the running system, so the chapter also acts as a bridge between what the stakeholders need and what the backend, the workers, and the user interface actually provide.

3.1 Stakeholder Analysis

Seed Bank serves two groups that have little in common. On one side are the people checking seed quality in a field or a warehouse. On the other are the people who build and look after the machine-learning model behind the product. Both groups depend on the same data, because the model that grades a farmer's photo is the same one a developer trained, registered, and evaluated. Naming each stakeholder and their goals early kept the requirements grounded, since a feature that helps one group should not break what the other relies on.

The farmer, or end user, is the main stakeholder. Their goal is to find out whether a batch of seeds is good before planting or buying it. They sign in from the web app or the mobile app, photograph a batch, and expect a quick good/bad report with a seed count and a good-rate, plus a history they can return to. Farmers often work on a phone and in Arabic, so the interface language and text direction matter as much as the result itself.

The AI developer keeps the product accurate over time. They train the detection and classification models, register the resulting weights, and run offline evaluations against labelled datasets to decide whether a new model is actually better than the one in use. Because they need to test a specific model on a real photo, they can pick which model runs at analysis time, which an ordinary user cannot do.

The administrator looks after the operational side. They choose which model serves traffic, manage users and their roles, and review the record of sensitive actions. An administrator can do

everything an AI developer can, plus the settings that change behaviour for everyone, so those actions are gated tightly and logged.

Table 3.1 summarises each stakeholder, their main goal, and the capabilities they lean on.

Table 3.1: Stakeholders, their goals, and the capabilities they depend on.

Stakeholder	Primary goal	Depends on
Farmer / end user	Check the quality of a seed batch in the field and review past results	Web/mobile capture, batch analysis, history, English/Arabic UI
AI developer	Train and validate models, then choose the right one	Model registry, datasets, offline evaluation, per-request model choice
Administrator	Control which model serves traffic and manage access	Model selection, user/role management, action log

3.2 Functional Requirements

The functional requirements describe what the system does, grouped by the part of the product they belong to. Each one corresponds to a capability in the running service, so they can be traced into the design and checked against it.

3.2.1 Accounts and access

A user can register with an email and password, confirm their address, and sign in. They can also sign in through Google. Once signed in, a user can manage their own account. Access is split across three roles (end user, AI developer, administrator), and each requirement below records the role allowed to perform it.

3.2.2 Analysis and history

The core action is submitting images for analysis. A user uploads a batch of images, optionally noting the seed type and supplier, and the system returns a good/bad verdict per seed along with a count and a good-rate for the batch. A faster mode trades some detail for a quicker turnaround on large batches. A user can also analyse a single image on its own. Around a batch, a user can list their batches, open one to see the aggregated scans, detections, statistics, and annotated images, export the results, download an image with the detection boxes drawn on it, and delete batches they no longer need.

3.2.3 Model work

AI developers and administrators look after the model. They register trained model weights along with their metadata, list and fetch them, and read how each model performed. They manage labelled datasets and run offline evaluations that score a model over a dataset. An AI developer can also choose which model runs for a given analysis, so a candidate model can be tried on a real photo before it takes over.

3.2.4 Administration

Administrators choose which model serves normal traffic, manage the user base by listing users and changing their roles, and read the log of sensitive actions.

Table 3.2 gives each functional requirement an identifier and the role permitted to perform it.

Table 3.2: Functional requirements with the role permitted to perform each.

ID	Requirement	Role
FR-1	Register with email/password and confirm the address	Public
FR-2	Log in and stay signed in across sessions	Public
FR-3	Sign in with Google	Public
FR-4	Submit a batch of images for analysis	End user
FR-5	Use the faster analysis mode for large batches	End user
FR-6	Analyse a single image on its own	End user
FR-7	List own batches and open one for detail	End user (own)
FR-8	View batch detail: scans, detections, statistics, and images	End user (own)
FR-9	Export a batch's results	End user (own)
FR-10	Download an annotated image with detection boxes	End user (own)
FR-11	Delete a batch	End user (own)
FR-12	Check service health and read runtime configuration	Public
FR-13	Register a model's weights and metadata; list and fetch models	AI developer
FR-14	Read per-model performance	AI developer
FR-15	Manage labelled datasets and run offline evaluations	AI developer
FR-16	Choose which model runs for a given analysis	AI developer
FR-17	Choose which model serves normal traffic	Administrator
FR-18	List users and change a user's role	Administrator
FR-19	Read the log of sensitive actions	Administrator

3.3 Non-Functional Requirements

Non-functional requirements describe how the system should behave while doing the work above. They cover the qualities a user or reviewer would notice if they were missing.

Performance. Analysis does not block the request. When a user submits a batch, the system accepts the images, starts the work in the background, and lets the client check back for the result. The images in a batch are processed independently rather than one after another, so a multi-image batch finishes faster than the sum of its parts. The faster analysis mode exists for cases where turnaround matters more than the extra detail.

Usability. The end-user surface is simple enough to use on a phone in the field. A user photographs a batch, waits a moment, and reads a clear good/bad result. History, exports, and annotated images are reachable in a few taps.

Internationalisation and RTL. The end-user interface works in both English and Arabic, including right-to-left layout. The layout flips correctly so dialogs, tables, and navigation read naturally in either language. The developer and admin pages are English-only, since farmers never reach them.

Security. Sign-in keeps users in their own data, and the three roles decide what each user can do, enforced on the server rather than just hidden in the interface. Sensitive actions are written to a log that can be reviewed but not edited after the fact.

Reliability. A batch moves through a clear set of states as it is processed. Several workers can handle the images of one batch at the same time without corrupting its state. If detection succeeds but classification later fails, the batch keeps the detections it already has instead of throwing them away, and a batch that cannot be matched to a model fails with a readable message.

Maintainability. The backend is layered so that each part has one job: request handling, business logic, and data access stay separate. Inference runs in a separate worker service rather than inside the web process, so the two can be changed and scaled on their own. Swapping the model that serves traffic is a registry-and-selection step, not a code change, which keeps the system easy to update as new crops and models are added.

Table 3.3 lists the non-functional requirements.

Table 3.3: Non-functional requirements.

ID	Attribute	Requirement
NFR-1	Performance	Analysis runs in the background; the images in a batch are processed in parallel.
NFR-2	Performance	A faster analysis mode is available for large batches.
NFR-3	Usability	The result is readable at a glance on a phone in the field.
NFR-4	Internationalisation	Full English/Arabic interface with right-to-left layout.

ID	Attribute	Requirement
NFR-5	Security	Three roles enforced on the server; sensitive actions logged.
NFR-6	Reliability	Clear batch state flow; concurrent workers cannot corrupt a batch; partial results are kept on classification failure.
NFR-7	Maintainability	Layered backend; inference in a separate worker; model swap is a selection step, not a code change.

3.4 Use Case Diagram and Actor Descriptions

The use cases group into three areas, following the same split used in the first-semester design: system utilities, history and statistics, and seed analysis. Figure 3.1 shows the actors and the main use cases.

System utilities cover the supporting calls that keep the service usable: a health check that reports whether the service is up, and a configuration call that lets a client read the current runtime settings.

History and statistics centre on viewing a batch in detail. Opening a batch pulls together its scan history, the individual seed detections, the aggregated statistics for the batch, and the annotated images into one view, so a user can see both the summary and the per-seed evidence behind it.

Seed analysis is the core of the system. A user analyses a batch of images in the standard mode, or uses the faster mode when a large batch needs a quicker turnaround. Both modes also work on a single image, which is useful for checking one sample or for tracing a result down to a single photo.

Three actors drive these use cases. The end user, a farmer or a person checking seed quality, runs analyses and reviews their own history. The AI developer does everything the end user can, and also trains models, registers them, runs offline evaluations, and chooses which model runs for a given analysis so a candidate can be tried on a real photo. The administrator does everything the developer can, and also chooses which model serves normal traffic, manages users and their roles, and reads the log of sensitive actions. These last actions change behaviour for everyone else, so they sit with the most-privileged actor.

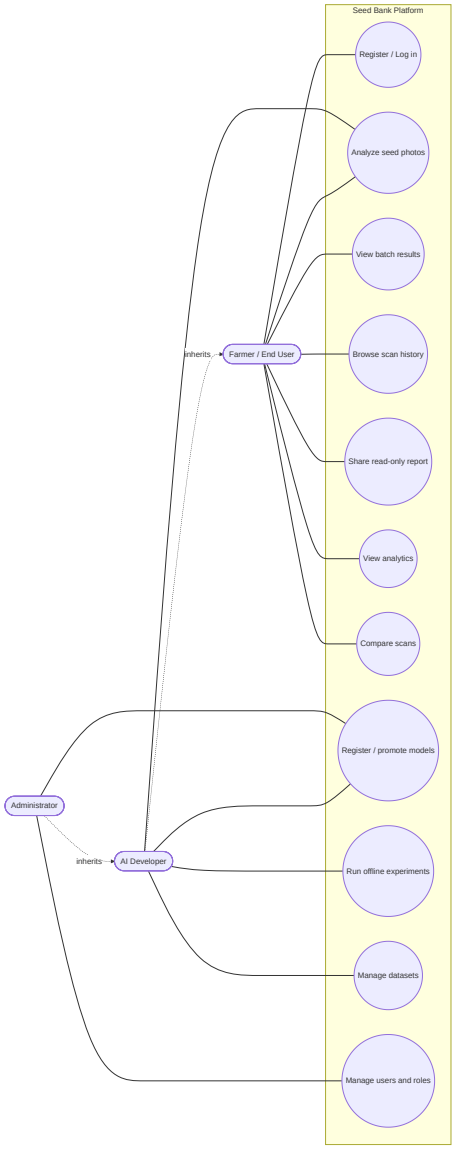

Figure 3.1: Use-case diagram (actors and main use cases).

3.5 Feasibility Study

Technical feasibility. The whole stack runs on commodity hardware. The backend, the inference worker, the database, the object store, and the supporting services come up together on a single developer machine, with demo data seeded in. A GPU is optional: routine development, the demo, and offline evaluation all run on CPU, and a GPU mainly helps when training new models. Every major component is a mature open-source project with good documentation, so the build does not rest on any unproven technology.

Economic feasibility. The cost case rests on open-source software and ordinary hardware. There are no licensing fees for any part of the stack, and the development and demo setup runs on one machine. A GPU is needed only for training or for heavy inference at scale; everything else runs on CPU. The object store and the model-tracking service are self-hosted alongside the

rest, so there is no recurring spend on managed cloud services to run or evaluate the system.

Operational feasibility. The product meets each group where they already are. Farmers reach it through a web app and a mobile app that uses the device camera, both in English or Arabic with right-to-left layout, so the interface does not assume English literacy or a desktop. Developers and administrators reach the same backend through a documented API and the role-gated web pages. Both clients talk to one shared API, so the system behaves the same across web, mobile, and direct API use, which keeps it practical to run and to hand over.

Chapter 4

System Design

This chapter describes how the Seed Bank prototype is put together: the overall architecture, the object model behind the inference pipeline, the main request flow, the relational schema, the screens the user sees, and the security posture. It also covers MultiSeedGen, the companion tool that generates the synthetic training data the detector learns from. The design keeps the parts that change for different reasons apart, so a new model, a new table, or a new screen can be added without disturbing the rest [25].

4.1 System Architecture

At a high level the prototype has five pieces. A React web client is the front end the user works with. A FastAPI backend exposes the API and runs the two-stage inference pipeline that turns a seed photo into a report [6]. A relational database (PostgreSQL) holds users, scan batches, and the per-seed results. An analytical database (ClickHouse) stores the inference metrics, scan statistics, and system telemetry for performance tracking. File storage keeps the uploaded images themselves, so the databases store references rather than image bytes.

Model inference is the heavy part of the work, so it does not run inside the request that the web client is waiting on. The API accepts a batch, records it, and hands the actual detection and classification off to a separate worker service. The worker pulls the image, runs the pipeline, and writes the results back to the database. This keeps the API responsive: a user submitting a batch gets an immediate acknowledgement instead of waiting on a slow forward pass, and several images can be graded in parallel by the worker.

Figure 4.1 places the system in its setting, showing the actors and the boundary they talk to.

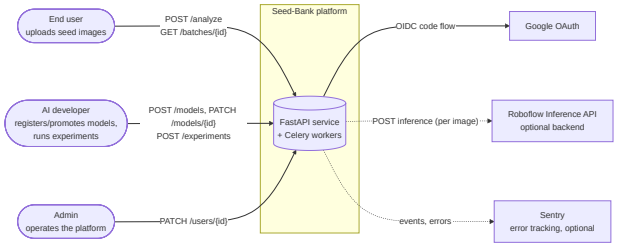

Figure 4.1: System context: the actors and the Seed Bank boundary they interact with.

Figure 4.2 shows the running pieces (web client, API, worker, databases, and file storage) and how they connect.

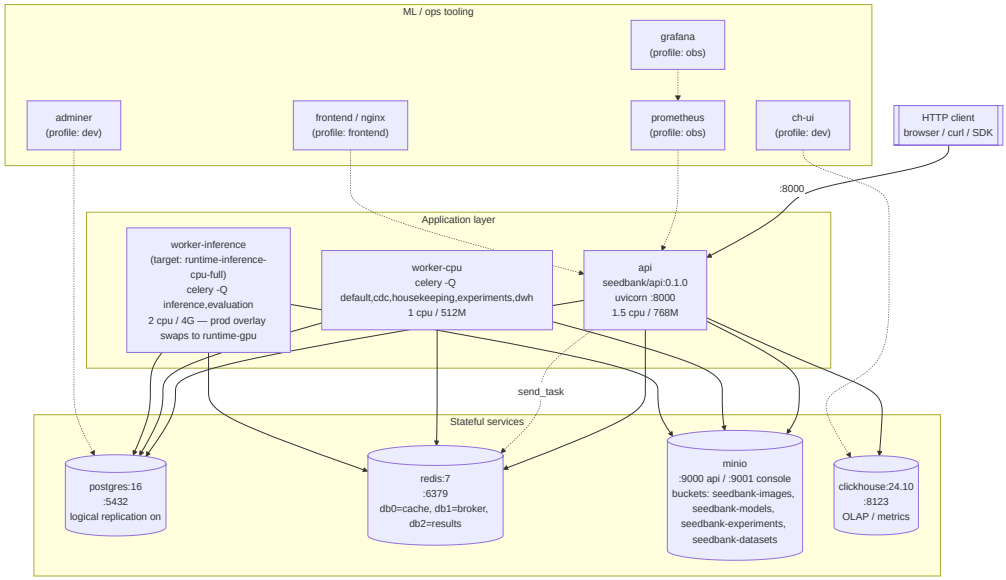

Figure 4.2: Container view: the web client, the API, the inference worker, the databases, and file storage.

The backend components are detailed in two parts. Figure 4.3 shows the request flow as it enters through middleware and is routed to core services. Figure 4.4 shows how these services interact with the repository layer and the underlying database ORM model.

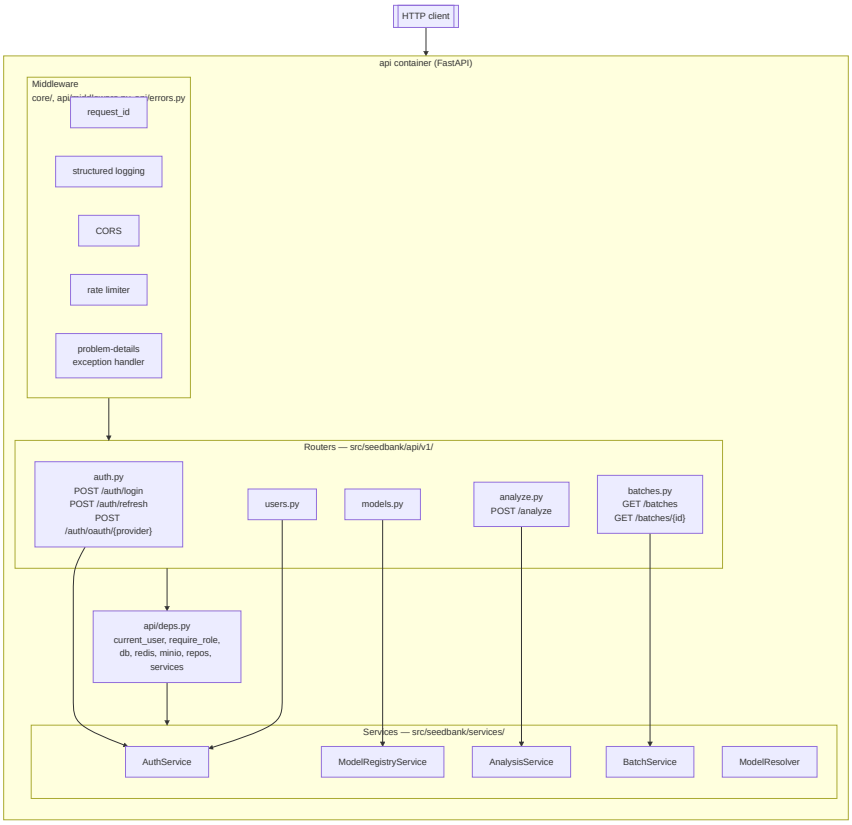

Figure 4.3: Backend request routing and services flow.

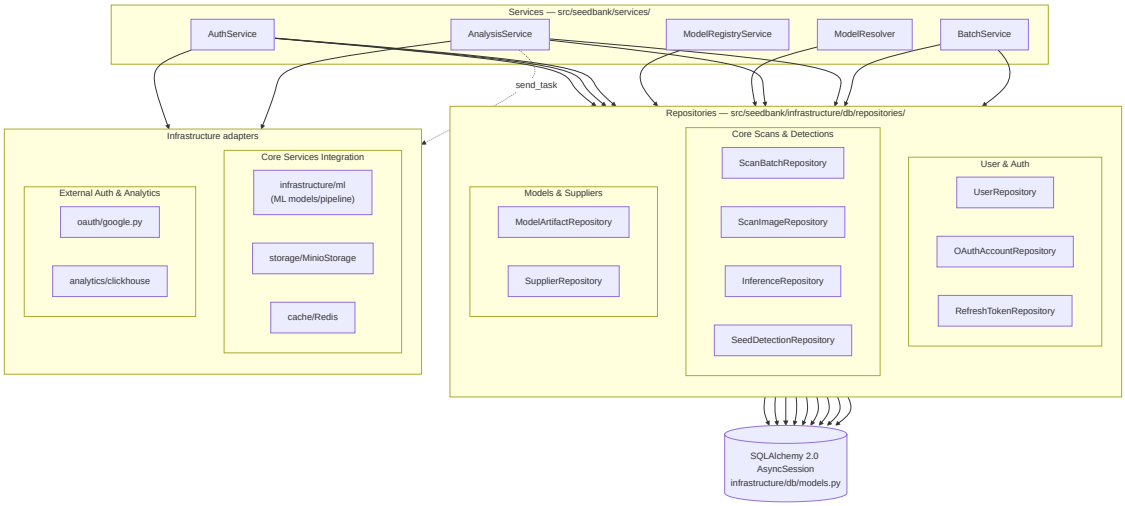

Figure 4.4: Backend data-access layer and repositories.

The worker side is similarly divided. Figure 4.5 shows the task orchestration and worker process loop. Figure 4.6 maps the detailed steps of the inference pipeline.

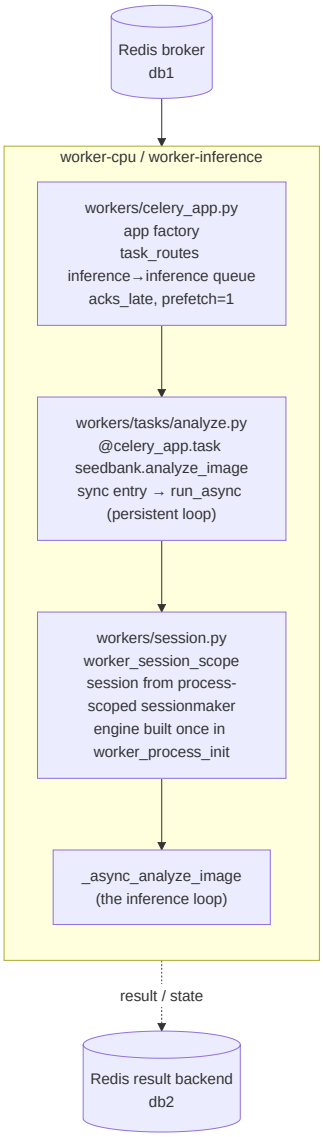

Figure 4.5: Inference worker task orchestration and dispatch.

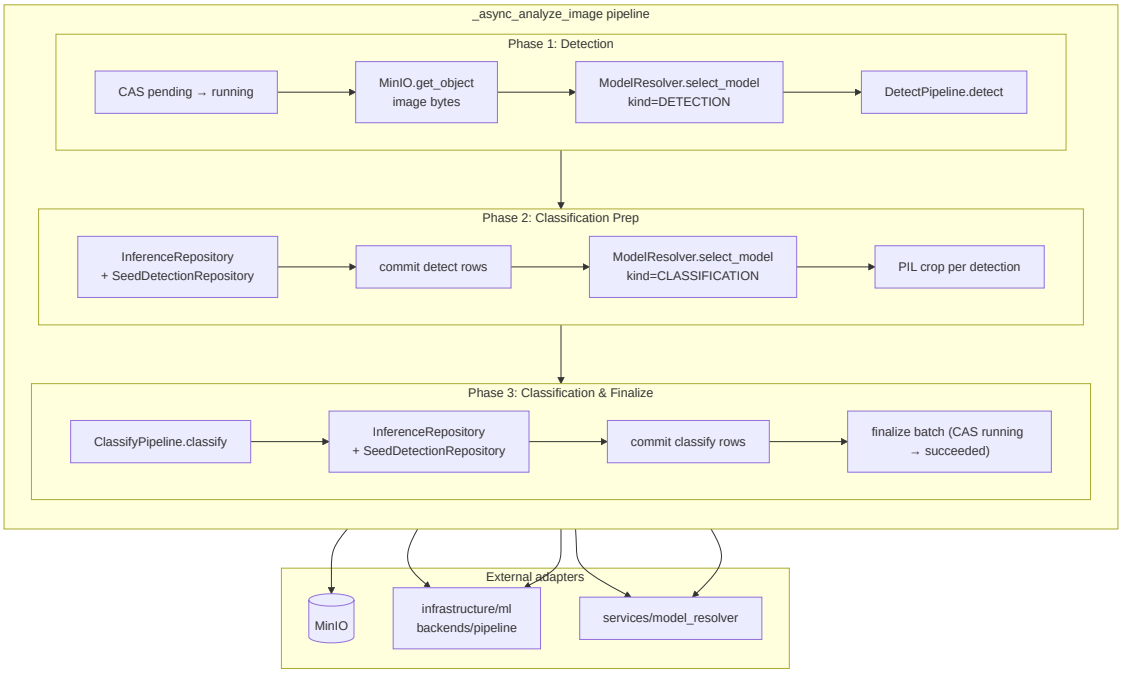

Figure 4.6: Inference worker detailed pipeline steps.

4.2 Class Diagrams

The object model is built around the scan and its results, shown in Figure 4.7. `ScanBatch` is the central entity. It tracks the processing status (`PENDING`, `PROCESSING`, `COMPLETED`) and aggregates results such as the total seed count and the average confidence score.

A `ScanBatch` has a one-to-many relationship to `ScanImage`: one batch contains several uploaded images. Each `ScanImage` in turn is associated with many `SeedDetection` records. A `SeedDetection` stores the granular output of the models for a single seed: the bounding box coordinates and a quality label of `GOOD` or `BAD`. A `User` entity owns the batches and ties each scan back to the account that created it.

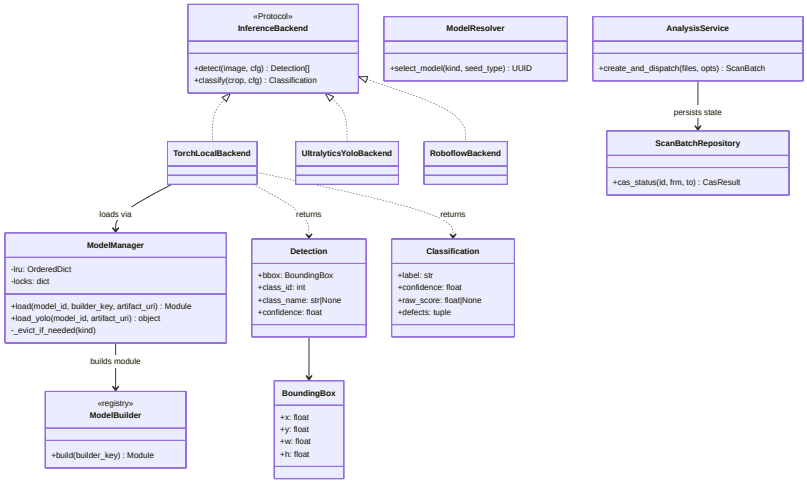

Figure 4.7: Class diagram: `ScanBatch` as the central entity, its images and per-seed detections, and the `User` that owns them.

4.3 Sequence Diagrams

The main interaction is batch analysis, shown in Figure 4.8. When a user submits a batch, the backend first identifies the user, then creates a new `ScanBatch` record in the `PROCESSING` state. It iterates through the uploaded files, saving each one to file storage and creating a matching `ScanImage` entry.

Each image then goes through the pipeline. The detector locates every seed in the image, and the backend records one `SeedDetection` per seed with its bounding box. The system then crops each detected seed from the source image and classifies it as good or bad. When every image is done it aggregates the statistics across the batch, updates the status to `COMPLETED`, and returns the structured report to the client.

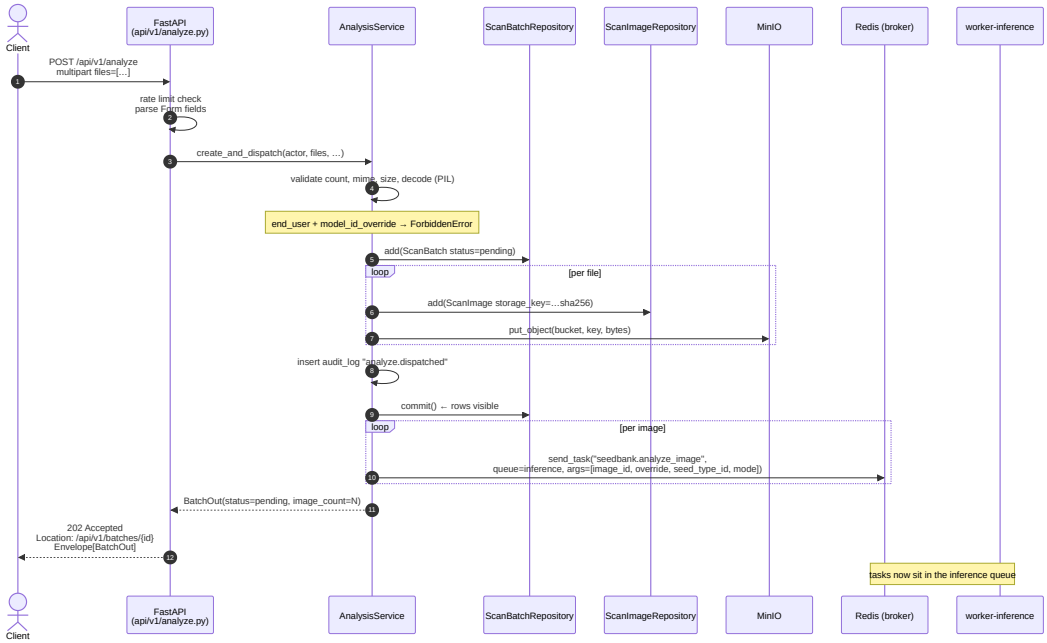

Figure 4.8: Batch-analysis sequence: identify the user, create the batch, save images, detect seeds, crop and classify each one, aggregate, and complete.

4.4 Database Entity-Relationship Diagram

The database schema is split into two logical areas to preserve printed legibility. Figure 4.9 shows the Core Platform and Analysis database tables (users, authentication details, audit log, scan batches, and detections). Figure 4.10 shows the Machine Learning Registry, datasets, and experiment tables.

The core scan hierarchy spans four central entities:

- `USER` manages identity and access, and owns the scans a person creates.
- `SCAN_BATCH` is the primary operational container. It tracks the progress of an analysis request (pending, processing, completed, or failed) and aggregates summary metrics (total seed count, bad-seed count, average confidence).
- `SCAN_IMAGE` records the metadata for every individual file in a batch, including its storage path and its pixel dimensions.
- `SEED_DETECTION` is the granular output of the models, storing a quality label (good or bad) and the normalized bounding box coordinates.

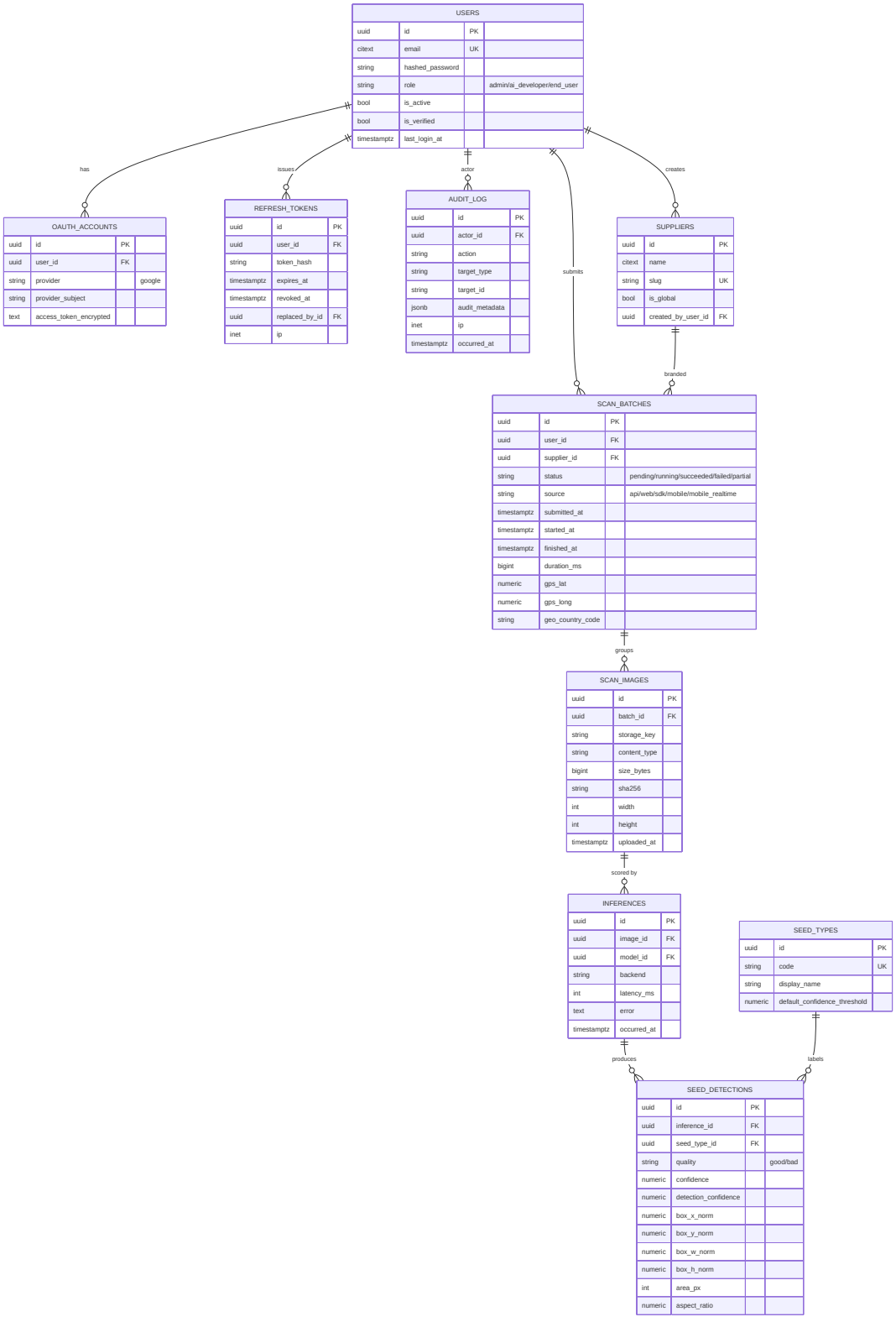

Figure 4.9: Core database schema: users, scans, and detections.

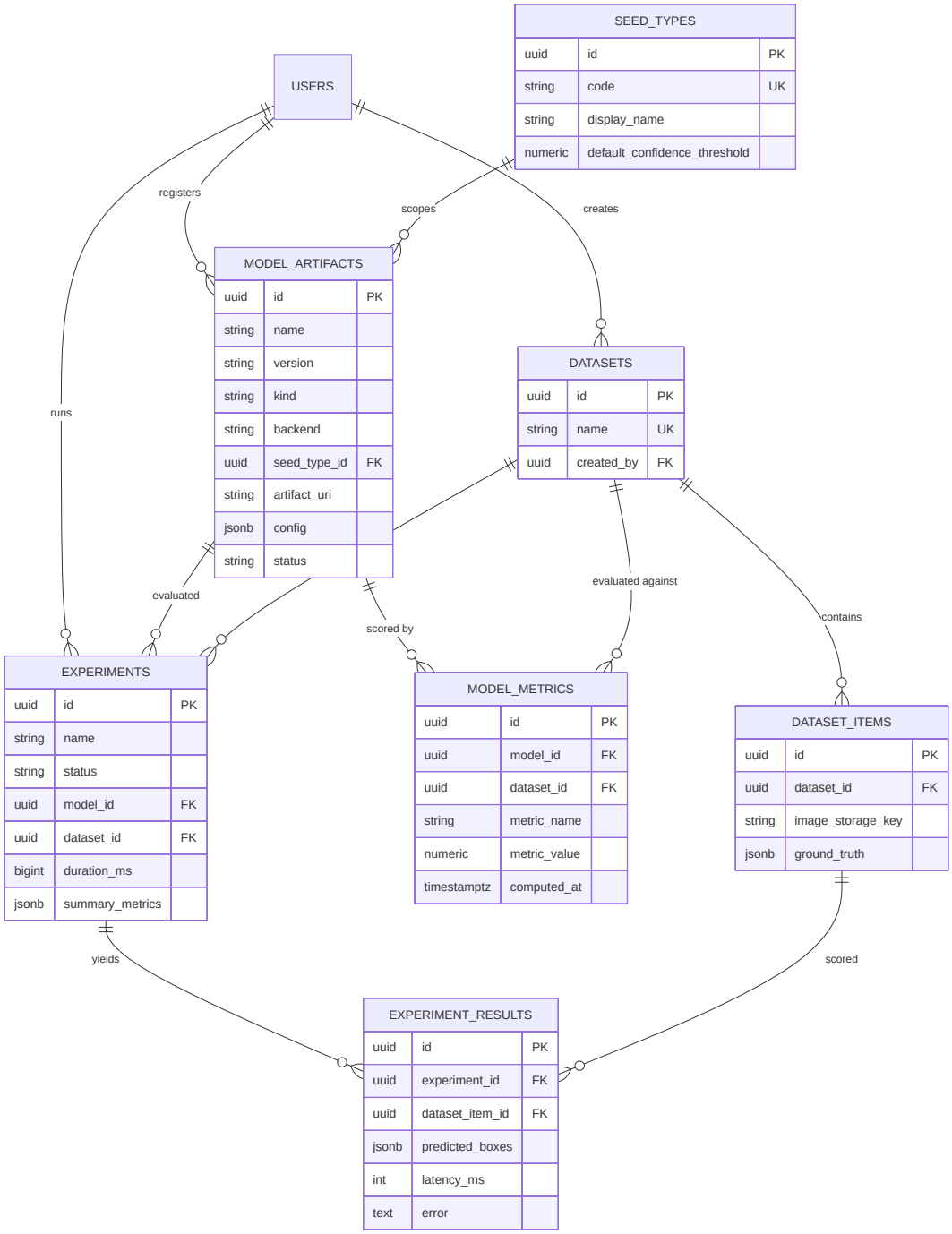

Figure 4.10: Machine learning database schema: models, datasets, and experiments.

4.5 MultiSeedGen Design

Training a seed detector needs many photographs of cluttered, overlapping seeds with a correct bounding box on every seed. These are slow to capture and tedious to annotate by hand. MultiSeedGen is the companion tool that manufactures this data. It takes photographs of single seeds and composites them into annotated multi-seed scenes whose labels are correct by construction, an instance of cut-and-paste synthesis [3]. The pipeline is shown in Figure 4.11.

The pipeline begins by segmenting each seed out of its source photo into a clean cut-out. A classical colour-and-brightness method is tried first, and a learned matting model is the fallback, either U2-Net [21] or Segment Anything [22]. The clean cut-outs form a seed pool.

Scene composition then paints seeds onto a background. It samples a number of seeds per scene and places each one with controlled placement, rejecting any position that overlaps an existing seed too much or that leaves a seed too hidden to label. Each placed seed gets its own augmentation (scale, rotation, and colour jitter) chosen so it does not invalidate the label. The seeds are composited onto varied backgrounds, from solid and textured fills to real inpainted photos. Some camera-style degradation (blur, noise, and compression) is added so the synthetic images resemble real phone photos, a form of domain randomization that helps a model trained on them transfer to real input [4], [23].

Because the tool knows exactly where it placed each seed, it records the exact bounding boxes and masks for every scene, accounting for seeds partly hidden behind others. Export then writes the dataset in the formats a trainer expects: YOLO boxes and segmentation polygons, and COCO annotations [20]. A leakage-safe split makes sure a source seed reserved for validation never appears in any training image, so the validation score is not inflated by the same seed showing up on both sides. The resulting detection and segmentation datasets are what train the Seed Bank detector.

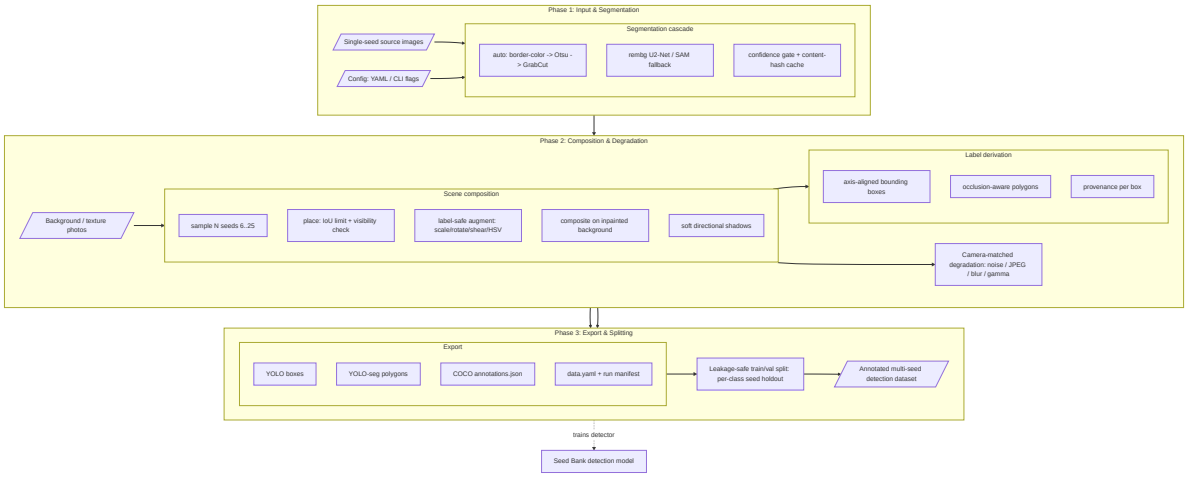

Figure 4.11: MultiSeedGen pipeline: segment single seeds, composite many onto varied backgrounds, augment per seed, record exact boxes and masks, and export detection and segmentation datasets.

4.6 User Interface Design (Wireframes and Mockups)

The prototype front end is a React web app that walks the user through the analysis workflow, from the dashboard to the per-seed results and the saved history. The screens below are the main views.

4.6.1 Dashboard

The dashboard is the central entry point for the workflow, shown in Figure 4.12. It presents a prominent drag-and-drop zone for uploading images, a few headline figures, and a list of recent scans, with a status indicator that confirms the backend is reachable.

[FIGURE PLACEHOLDER]

figures/web-dashboard.png

Dashboard: the drag-and-drop upload zone, headline figures, and recent scans.

Figure 4.12: Dashboard: the drag-and-drop upload zone, headline figures, and recent scans.

4.6.2 Analysis and results workspace

The analysis results workspace is where the user verifies the predictions, shown in Figure 4.13. The processed image is displayed with color-coded bounding boxes, green for good seeds and red for bad seeds, so the overall quality reads at a glance. A panel alongside the image gives aggregate metrics, including the total detection count and the purity percentage. Below it, a scrollable per-seed list details each detection with its confidence score. A tabbed bar lets the user move between the images in a multi-image batch without losing context.

[FIGURE PLACEHOLDER]

figures/web-batch-detail.png

Analysis and results workspace: color-coded bounding boxes (green good, red bad), the per-seed list, and the purity panel.

Figure 4.13: Analysis and results workspace: color-coded bounding boxes (green good, red bad), the per-seed list, and the purity panel.

4.6.3 Reporting and export

The reporting view generates an exportable session summary, shown in Figure 4.14. It opens with high-level metrics, including a good versus bad seed count shown as a pie chart and a per-image breakdown table that points out skewing samples. Each image is detailed with its

annotated bounding boxes, and a final table lists every detected seed with a thumbnail, its quality label, and its confidence score, so a reviewer can check the results seed by seed.

[FIGURE PLACEHOLDER]

figures/web-analyze.png

Reporting and export view: high-level metrics, the good-versus-bad pie chart, the per-image breakdown, and the per-seed table.

Figure 4.14: Reporting and export view: high-level metrics, the good-versus-bad pie chart, the per-image breakdown, and the per-seed table.

4.6.4 Scan history

The scan-history view is the repository for previously run analyses, shown in Figure 4.15. A statistical panel at the top aggregates global figures, such as the total number of batches processed and the cumulative seed count. Below it, each past session is a chronological card showing the timestamp, the final status, and a good-versus-bad breakdown with the average confidence. A filter sorts records by status, and a ”View Details” action reopens the full visual data and report for any archived session.

[FIGURE PLACEHOLDER]

figures/web-login.png

Scan history: the global statistics panel and the chronological session cards with filtering and per-session detail.

Figure 4.15: Scan history: the global statistics panel and the chronological session cards with filtering and per-session detail.

4.7 Security Design Considerations

Security is brief and standard for a prototype of this size. Sessions use tokens: the user signs in and the backend issues a short-lived access token that the client sends with each request. Passwords are never stored in plaintext; only their hashes are kept, so a database leak does not expose a usable credential. Access is role-based, so an ordinary user can analyze and see only their own history, while administrative and model-management actions are gated to the roles that are allowed to perform them.

Chapter 5

Implementation

The previous chapter described what the system should do and how its pieces fit together. This chapter is about how those pieces were actually built: the setup that lets the whole platform run on a laptop, the implementation choices that turned the design into working code, and a few screenshots and generated samples that show the result. The synthetic-data tool, MultiSeedGen, gets the most space, because it is where most of the engineering effort went and it is the part of the project we can defend in the most detail.

5.1 Implementation environment and setup

The whole system is containerized with Docker [8]. The goal was that a fresh checkout reaches a working stack with a single command, without anyone installing a database or a Python toolchain on the host by hand. Docker gives us an environment that behaves the same on a personal laptop, a lab machine, or a server, which removes the usual ”it works on my machine” problem and makes it easy for a new team member to get started.

There are four primary components. The frontend is a React application [7] that the user interacts with. The backend is a FastAPI service [6] that takes an image, runs the models, and returns the results. PostgreSQL stores the transactional records so a user can come back later and see the history of what they scanned. Alongside PostgreSQL, a ClickHouse instance acts as the analytics data warehouse, storing detailed inference telemetry and performance metrics for reporting and evaluation. The frontend talks to the backend over a simple HTTP API, and the backend communicates with the databases and object storage, so each part can be developed and tested on its own.

Figure 5.1 shows how the containers connect.

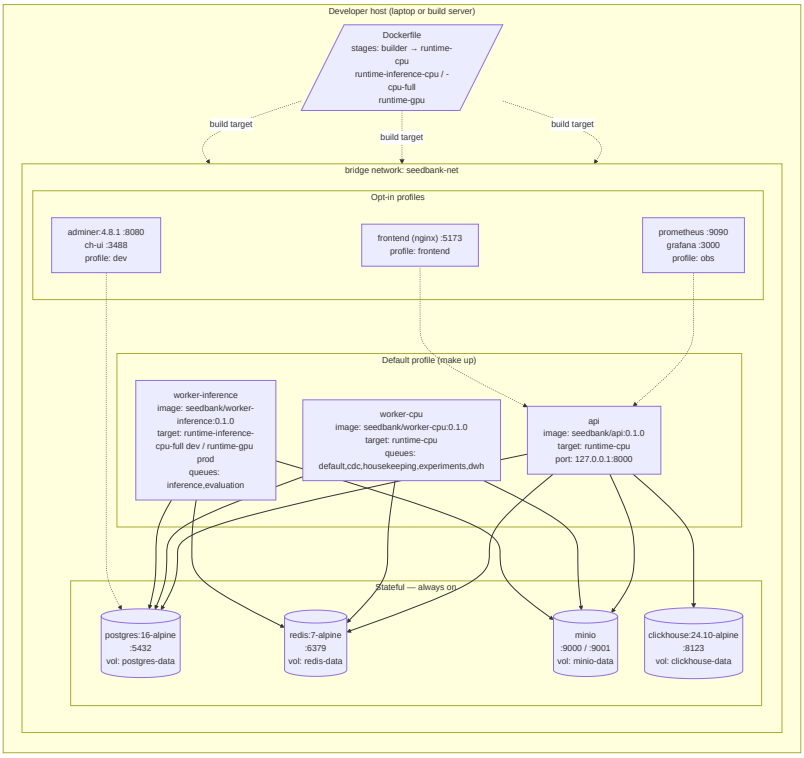

Figure 5.1: Deployment view: the React frontend, the FastAPI backend, the PostgreSQL database, and the ClickHouse analytics warehouse.

5.2 Key implementation details

5.2.1 The two-stage detection and classification pipeline

Checking a photo of seeds is really two separate problems, and the implementation treats them as two models run one after the other. The first stage is detection. An object detector scans the whole image and returns, for every seed it finds, a bounding box, a confidence score, and the crop type. We support two crops, coffee and maize, so the detector tells us where each seed is and which of the two it is. One photo therefore becomes a set of located, labelled seeds.

The detector is a Faster R-CNN [14] with a ResNet-50 backbone [18] and a Feature Pyramid Network [15]. The feature pyramid lets the model find seeds at different sizes in the same image, which matters because a photo can hold a hundred seeds at slightly different distances from the camera. Faster R-CNN is the accurate option. For situations where speed matters more than the last point of accuracy, such as a conveyor-belt feed or a phone in the field, there is an optional YOLO mode [16], [17] that runs the detection in a single pass and is much faster, at some cost in precision.

The second stage is classification. For each detected seed, the backend crops that bounding box out of the original photo and sends the crop to a quality classifier, which returns good or bad with a confidence. The classifier is a ResNet-18 [18], chosen because it is small and fast enough to run over the many seeds in a single batch. We made two changes to the standard

network. First, we inserted a CBAM attention block [5], which pushes the model to focus on the parts of a seed that signal a defect, such as a dark spot or a crack, instead of the overall shape. Second, instead of a single global-average pool we use a hybrid of global-average and global-max pooling: the average pool captures the general appearance of the seed and the max pool keeps sharp local features that an average would smooth away. There is one classifier per crop, a coffee classifier and a maize classifier, so detections are grouped by crop type and each group goes to its matching model.

Keeping detection and classification as two separate models, rather than one model that tries to do everything, is the central design decision. It lets each stage use an architecture suited to its job: a detector tuned to tell crops apart and locate them, and a smaller classifier tuned to the fine-grained difference between a healthy and a defective seed of the same crop.

5.2.2 Model management and evaluation

The backend also has a small amount of supporting machinery around the models. It can register and swap the trained weights without code changes; it can run an offline evaluation of a model against a held-out set, storing each run's metrics in PostgreSQL alongside a Markdown report in object storage so that versions can be compared before one is put into use; and it runs the actual inference in a background worker so that uploading a batch of images does not block the rest of the application while the models run. This part stayed simple on purpose, since the prototype only needs to serve and compare a handful of models.

Choosing which model actually runs for a given request is the job of the model resolver. Given a segment — the kind of model needed (detection or classification) and, optionally, the seed type — it resolves the model promoted to `production` for that exact segment; if none has been promoted and a seed type was supplied, it falls back to the global, seed-type-agnostic production model for that kind; and if nothing is routable it reports that no model is ready rather than failing silently. The decision is deterministic and stateless — there is no weighted routing and no per-user bucket — so the same request always resolves to the same model, and a developer can override the choice with an explicit model on a single request. Figure 5.2 shows the decision.

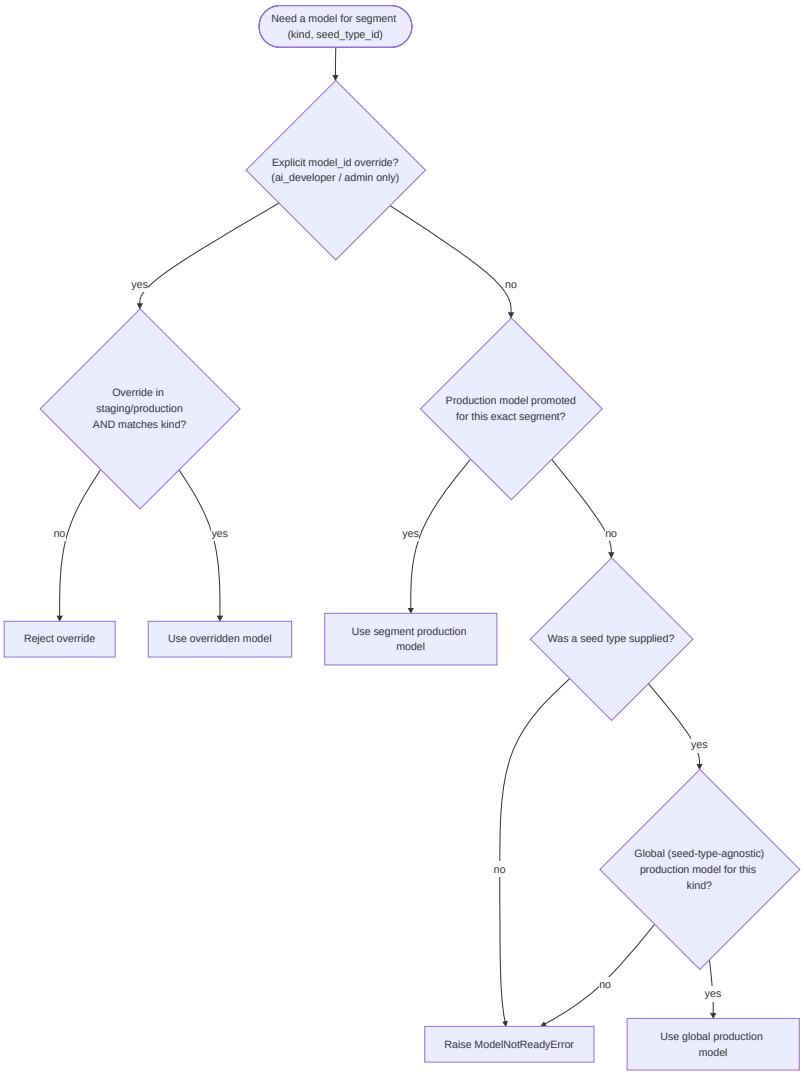

Figure 5.2: Model resolution: the resolver selects the model promoted to production for a segment, falls back to the global production model when only that is available, or reports that no model is ready.

5.2.3 Data warehouse and analytics telemetry (ClickHouse)

To track the real-world performance of the grading system, the platform implements an online analytical processing (OLAP) star schema in ClickHouse. While PostgreSQL manages transactional and relational data (such as user credentials, scan batches, and model status), ClickHouse is dedicated to high-performance analytics, storing scan telemetry, inference latency, model accuracy over time, and geographic distribution metrics.

The analytics database schema consists of dimension tables (such as `dim_user`, `dim_model`, and `dim_seed_type`) and fact tables (such as `fact_inference`, `fact_detection`, and `fact_scan_batch`). Data is synchronized from PostgreSQL using application-level dual-writes rather than log-based change data capture (CDC). After each successful transactional commit, a Celery worker dispatches an idempotent insert to ClickHouse via the `AnalyticsRepository` using the async driver `clickhouse-connect`.

Because ClickHouse uses the `ReplacingMergeTree` table engine, duplicate inserts collapse on their sort key during merge cycles. This makes at-least-once delivery from the Celery tasks completely safe against duplicates, ensuring high consistency without complex distributed locking. Callers query this database on async read paths to drive the dashboard statistics, leaderboards, and offline performance tracking.

5.2.4 MultiSeedGen: the synthetic-data pipeline

Training a detector needs labelled images that contain many seeds at once, with a box around each one. Those are far harder to collect than the single-seed photos in the public datasets: someone would have to lay out dozens of seeds, photograph them, and then hand-draw and label a box around every seed in every image. MultiSeedGen avoids that work. It cuts individual seeds out of single-seed source photos, pastes many of them onto realistic backgrounds, and exports a fully labelled multi-seed detection dataset. Because the tool places every seed itself, it already knows exactly where each seed is and what shape it has, so the bounding box and the segmentation mask are known precisely. The annotations come for free, with no manual labelling.

The hardest step is segmentation: cutting one seed cleanly out of its source photo, including the soft edges. MultiSeedGen runs this as a cascade that starts cheap and escalates only when needed. It first tries classical methods, a colour-distance threshold against the background followed by GrabCut, which are fast and good enough for seeds on a plain background. When those produce a poor mask, it falls back to rembg, a U^2-Net model [21], and for the hardest cases with feathered or low-contrast edges it uses Segment Anything [22]. A cut-out whose mask scores too low is dropped rather than pasted in with a bad outline, so a poor segmentation never becomes a bad label.

With clean cut-outs in hand, the tool composites a scene. It pastes many seeds onto a background, the cut-and-paste idea from Dwibedi et al. [3], varying the backgrounds so the detector does not just learn one surface. To stop the synthetic images from looking too clean and too similar to each other, MultiSeedGen applies domain-randomized augmentation: it randomizes the geometry of each seed, its rotation, scale, and placement, and the lighting and colour of the scene [4], [23], [24]. The aim is for a model trained on these images to transfer to real photos, since it has seen a wide range of arrangements and conditions rather than one narrow style.

Two details keep the resulting dataset trustworthy. First, the split between training and validation is leakage-safe: a given source seed is assigned to either training or validation, never both, so the same physical seed cannot show up in a training image and a validation image at once. Without this, the validation score would be inflated, because the model would effectively be tested on seeds it had already seen. Second, the datasets export to both COCO and YOLO formats [20], the two formats the detection models expect, so the generated data drops straight into training.

5.3 Sample screenshots and output samples

This section shows the running system and the artefacts it produces.

The end product of the two-stage pipeline is an annotated image. Figure 5.3 shows the detector's boxes drawn back over a photo, each one labelled with the classifier's good or bad verdict for that seed. This is the overlay the frontend shows the user after a scan.

[FIGURE PLACEHOLDER]

figures/sample-detection-output.png

Annotated output: detection boxes with a per-seed good or bad quality label from the classifier.

Figure 5.3: Annotated output: detection boxes with a per-seed good or bad quality label from the classifier.

Figure 5.4 is a scene generated by MultiSeedGen, with the bounding boxes drawn back from the exported labels. This is the kind of image the tool produces by the thousand to train the detector.

[FIGURE PLACEHOLDER]

figures/msg-sample-scene.png

A MultiSeedGen scene: many cut-out seeds composited onto a background, with bounding boxes drawn from the exported labels.

Figure 5.4: A MultiSeedGen scene: many cut-out seeds composited onto a background, with bounding boxes drawn from the exported labels.

Figure 5.5 is the segmentation tuner from the MultiSeedGen interface, which shows the kept and skipped cut-outs side by side so we can see which seeds a given method dropped, and why, before running a full generation.

[FIGURE PLACEHOLDER]

figures/msg-seg-tuner.png

MultiSeedGen segmentation tuner: kept and skipped cut-outs shown together so the segmentation quality can be checked before a full run.

Figure 5.5: MultiSeedGen segmentation tuner: kept and skipped cut-outs shown together so the segmentation quality can be checked before a full run.

Chapter 6

Testing and Evaluation

This chapter reports how the system was tested and, more importantly, what the trained models actually do on real data. It opens with a short account of the software test suite, then spends most of its length on the experiments: the datasets, the detection results, the quality-classification results across the model variants, and the measured latency of the running prototype. It closes with the limitations the numbers expose.

6.1 Testing Strategy

The backend carries most of the software tests because it is where a wrong change does the most damage. The suite is layered. Unit tests check single functions in isolation with the database, the object store, and the model backends replaced by fakes, so they run in milliseconds and cover the pure logic: request and response validation, the batch state transitions, and the authentication token paths. Integration tests run against real PostgreSQL and ClickHouse instances started for the test run and torn down afterwards, so both the transactional repository code and the analytical data-warehouse code are checked against the database engines they actually use rather than mocks that might disagree with real SQL. End-to-end tests drive the assembled HTTP application and assert on status codes and response bodies, exercising routing, authentication, and serialization together. On top of these sits a smoke test that boots the full stack, registers a model, submits an analyze request, and waits for the batch to finish, which is the closest check to a real deployment.

The web client has component tests, including one that switches the interface to Arabic and asserts that the layout flips to right-to-left and the choice persists. MultiSeedGen has two checks that matter for a data generator: a golden-output test that pins the exact bytes of a generated dataset so any change to the output is caught, and a determinism test that asserts the same configuration and random seed always produce identical output regardless of how many worker processes run.

Table 6.1 lists representative cases from across the suite. The result column reports pass

where the behaviour matches the expected contract.

Table 6.1: Representative software test cases.

ID	Area	Scenario	Expected	Result
T-01	Auth	Login with valid credentials.	An access token and a refresh token are issued.	Pass
T-02	Auth	A used refresh token is presented again.	The reused token is rejected and the session is invalidated.	Pass
T-03	Analyze	A batch of valid images is uploaded.	The batch is created and one analysis task is queued per image.	Pass
T-04	Analyze	A file that is not a real image is uploaded.	Validation fails and no batch is stored.	Pass
T-05	Pipeline	Detection succeeds but classification fails on one image.	Detections are kept and the batch finishes partial, not failed.	Pass
T-06	Generator	MultiSeedGen runs twice with the same seed and different worker counts.	Both runs produce byte-identical output.	Pass
T-07	Generator	A train/validation split is built from the seed pool.	No source seed appears in both partitions.	Pass

6.2 Datasets

The models were trained and evaluated on data drawn from four sources, chosen so the system would see more than one capture condition. Public seed datasets are usually shot in a controlled lab, and a model trained only on that kind of image tends to fail on a photo taken with a phone under ordinary light. Mixing sources was the main defence against that.

The first source is the NAGAR maize dataset, built on the IIIT-Hyderabad corn seed benchmark [26]. We used the balanced variant and removed noisy labels to improve quality. For the classifier the data is reduced to two classes: good, which is healthy pure corn, and bad, which combines the broken, discoloured, and silkcut defects.

The second source is a maize dataset from Roboflow with 26,350 images [27]. Unlike the lab-controlled NAGAR set, it spans a wide range of lighting and texture and is already split into binary good and bad classes. Combining it with NAGAR produced what we call the hybrid dataset.

The third source is a smaller but densely annotated detection set of 600 high-density images used to train and stress-test the localization models [28]. Every seed carries a bounding box, and the boxes are labelled across seven maize classes: broken (2,927), damage (3,766), fungus

(1,510), immature (1,721), shriveled (1,618), weeveled (1,835), and healthy (1,706).

The fourth source is a coffee parchment dataset, the dried endocarp of the coffee bean, with good and bad classes. It was added to check whether the design generalizes to a second crop with a different shape and spatial arrangement.

6.3 Object Detection Results

The first stage of the pipeline locates each seed. We evaluated Faster R-CNN [14] in four configurations, with and without a CBAM attention block [5], on maize alone and on maize and coffee together. Detection quality is reported as mean average precision at three intersection thresholds: mAP@50, the stricter mAP@75, and the COCO average over 0.50 to 0.95. Table 6.2 collects the results.

Table 6.2: Faster R-CNN detection results across configurations.

Configuration	**mAP@50**	**mAP@75**	**mAP .5:.95**	**Time (s)**
Faster R-CNN, maize only	0.9780	0.6574	0.5903	0.1511
Faster R-CNN + CBAM, maize only	0.9768	0.6576	0.5867	0.0621
Faster R-CNN, maize & coffee	0.9838	0.8015	0.7038	0.1038
Faster R-CNN + CBAM, maize & coffee	0.9835	0.8020	0.7211	0.1106

Two patterns stand out. On the single-class maize task, adding CBAM did not help; the standard and attention models score within a thousandth of each other on mAP@50, so the extra complexity bought nothing on an easy, single-class set. On the harder two-crop task the picture changes. Moving from maize-only to mixed maize-and-coffee data lifted every metric, and the jump is largest on the strict thresholds: mAP@75 rises from about 0.66 to about 0.80 and the COCO average from 0.59 to 0.70. Adding more variety to the training data helped the detector more than any architectural change did. On that mixed set CBAM gave a small but real gain on the strictest metric, nudging the COCO average from 0.7038 to 0.7211, which fits the idea that attention earns its cost on harder, more varied inputs.

For deployments where speed matters more than the last point of accuracy, we also trained a one-stage YOLOv8 detector [16], [17]. It reaches an mAP@50 of 0.941 with a precision of 0.939 and a recall of 0.941, at roughly 5.5 ms per image, which is far faster than Faster R-CNN and is why the prototype offers it as a fast mode.

6.4 Seed Quality Classification Results

The second stage takes each detected seed and labels it good or bad. The classifier is a ResNet-18 [18] that we extended in four steps to measure the effect of each change: V1 is the plain baseline, V2 adds a CBAM attention block [5], V3 adds global max pooling alongside the usual average pooling, and V4 adds a stride change in the final stage. We trained this family twice, once on the original NAGAR data and once on the hybrid set that adds the Roboflow images. Table 6.3 shows the headline comparison between the baseline on each.

Table 6.3: Baseline classifier on the NAGAR and hybrid datasets.

Model	Training data	Acc.	F1	Time
Baseline ResNet-18	NAGAR only	96.19%	0.9626	70.05 s
Hybrid ResNet-18	NAGAR + Roboflow	97.39%	0.9745	143.50 s

6.4.1 Validation performance

On held-out validation data the four variants all score highly, which is expected because validation data comes from the same distribution as training. Tables 6.4 and 6.5 report the variants on the NAGAR and hybrid sets.

Table 6.4: Classifier variants on the NAGAR dataset (validation).

Variant	F1	Combined	Precision	Recall
V1 Baseline ResNet-18	0.9626	0.9623	0.9768	0.9489
V2 + CBAM	0.9760	0.9757	0.9869	0.9653
V3 + CBAM + GMP	0.9758	0.9755	0.9821	0.9696
V4 + CBAM + GMP + Stride	0.9734	0.9730	0.9797	0.9671

Table 6.5: Classifier variants on the hybrid dataset (validation).

Variant	F1	Combined	Precision	Recall
V1 Baseline ResNet-18	0.9779	0.9776	0.9846	0.9714
V2 + CBAM	0.9789	0.9786	0.9846	0.9732
V3 + CBAM + GMP	0.9792	0.9789	0.9822	0.9763
V4 + CBAM + GMP + Stride	0.9767	0.9764	0.9839	0.9696

Three things are visible. Adding the Roboflow data raises the baseline F1 by roughly 1.5 points, from 0.9626 to 0.9779, before any architecture change. CBAM (V2) gives the largest single jump in precision on both sets. And V3, which adds global max pooling, reaches the

highest recall, which makes sense because max pooling responds to sharp local defects and so catches more of the bad seeds.

6.4.2 Performance on unseen data

Validation numbers flatter a model. The real test is data the model has never seen, drawn from a different distribution, where domain shift bites. Tables 6.6 and 6.7 report the same variants on an out-of-distribution test set.

Table 6.6: Classifier variants trained on NAGAR, evaluated on unseen data (test).

Variant	Acc.	F1	Precision	Recall	Combined
V1 Baseline	0.7722	0.5832	0.7568	0.4746	0.6766
V2 + CBAM	0.7920	0.5892	0.7768	0.4746	0.6906
V3 + GMP	0.7722	0.5162	0.7763	0.3866	0.6442
V4 + Stride	0.7821	0.5349	0.8125	0.3987	0.6585

Table 6.7: Classifier variants trained on the hybrid dataset, evaluated on unseen data (test).

Variant	Acc.	F1	Precision	Recall	Combined
V1 Baseline	0.8062	0.7477	0.6328	0.9136	0.7769
V2 + CBAM	0.8130	0.7499	0.6470	0.8918	0.7815
V3 + GMP	0.7725	0.7259	0.5841	0.9588	0.7492
V4 + Stride	0.8318	0.7687	0.6769	0.8894	0.8003

The drop from validation to test is large and worth stating plainly. A model trained on NAGAR alone falls from an F1 near 0.97 on validation to about 0.58 on unseen data, because it learned the lab look of its training set. Training on the hybrid set recovers most of that loss: the peak test F1 rises from 0.5892 to 0.7687, and recall climbs as high as 0.9588 for V3. The lesson is the one the detection results pointed at too. Diverse training data, not a cleverer architecture, is the main defence against domain shift. Among the hybrid-trained variants, V4 is the most balanced, with the highest test accuracy (83.18%) and the best combined score (0.8003).

6.4.3 The second crop

To check that the design carries to a different seed, we ran the same four variants on the coffee dataset. Table 6.8 reports the test results.

Table 6.8: Classifier variants on the coffee dataset (test).

Variant	F1	Combined	Precision	Recall
V1 Baseline ResNet-18	0.9056	0.8984	0.8898	0.9219
V2 + CBAM	0.9023	0.8957	0.8988	0.9058
V3 + CBAM + GMP	0.9101	0.9028	0.8877	0.9335
V4 + CBAM + GMP + Stride	0.9000	0.8921	0.8803	0.9206

V3 is the best variant on coffee, with the highest F1 (0.9101) and recall (0.9335), the same combination of attention and max pooling that did best on maize. That the same design wins on two crops with different shapes is a small piece of evidence that the attention blocks belong in the later layers of the network, where they filter high-level features rather than low-level edges.

6.5 System Performance and Latency

Accuracy only matters if the system is fast enough to use. We measured the prototype end to end on commodity hardware. A batch of three images containing more than 200 seeds was processed in about 1.78 seconds, counting the upload, the database writes, and the generation of the result report. That is fast enough for a quality-control operator to work through samples without waiting on the software.

Broken down by stage, the detector averages about 103.8 ms per image, and the per-seed quality classifier runs in roughly 0.50 to 0.67 ms per crop, so the classification stage is cheap even when an image contains a few hundred seeds. The YOLOv8 fast mode brings detection down to about 5.5 ms per image for deployments that prioritize speed. Table 6.9 collects these figures.

Table 6.9: Measured latency of the running prototype.

Stage	Latency
Full batch (3 images, 200+ seeds, end to end)	$\approx$ 1.78 s
Detection per image (Faster R-CNN)	$\approx$ 103.8 ms
Detection per image (YOLOv8 fast mode)	$\approx$ 5.5 ms
Quality classification per seed	$\approx$ 0.50–0.67 ms

6.6 Limitations and Known Issues

The results above also mark the limits of the work. The clearest one is the synthetic-to-real domain gap. MultiSeedGen produces realistic composited scenes, but a detector trained on

them learns the statistics of synthetic images, and those never match real trays exactly [4]. The generator narrows the gap with domain-randomized lighting and degradation, but closing it for a given crop still means validating the trained detector on genuine field photographs before trusting it [1].

The classification results show the same gap from the other side. The fall from a validation F1 near 0.97 to a test F1 near 0.58 for the NAGAR-only models is a direct measurement of domain shift, and while the hybrid data recovers much of it, the test scores are still well below the validation scores. Any deployment to a new capture setting should expect to collect and label some local data and retrain, rather than assume the validation numbers will hold.

Finally, this is a prototype rather than a finished product. It grades two crops from top-down photographs taken in reasonable light, and it has not been hardened or trialled in the field. The next chapter sets out what would have to change to move it past that point.

Chapter 7

Conclusion and Future Work

This chapter closes the report. It restates what was built and maps it back to the objectives from Chapter 1, records the lessons that came out of building it, and sets out the work that would carry the system from a prototype toward something a seed cooperative could run for a real season.

7.1 Summary of achievements

The project set out to build an automated seed-quality grading prototype: a user photographs a batch of seeds, and the system finds each seed and judges its quality, then reports aggregate figures such as seed count and the proportion of good seeds. That prototype exists and runs end to end for two crops, coffee and maize.

The core of the system is a two-stage vision pipeline. A detection model scans the whole image and returns a bounding box and a crop label for every seed it finds. Each detected box is then cut from the image and passed to a classifier that decides whether the seed is good or bad [1]. The detector is a Faster R-CNN with a ResNet-50 backbone and a Feature Pyramid Network [14], [15], [18]. The classifiers are ResNet-18 backbones with a Convolutional Block Attention Module added after the final residual stage [5], [18], with a separate classifier trained for each crop. Splitting detection from classification means the two halves can be measured and improved on their own, which mattered through the project because the failures in each half had different causes.

We evaluated both stages on real data, not only on synthetic samples. The detector was scored with mean average precision and the classifiers with precision, recall, and F1, against held-out images that the models never saw during training. The full set of numbers, including the per-class breakdown and the confusion matrices, is reported in the evaluation in Chapter 6. The headline is that a two-stage detect-then-classify pipeline trained largely on data we generated ourselves reaches usable accuracy on genuine photographs of coffee and maize seeds. The crop classification borrows the transfer-learning approach that has worked across plant-image tasks [2], with backbones pre-trained on ImageNet [19] and fine-tuned on our seed images.

7.1.1 MultiSeedGen: removing the annotation bottleneck

The companion tool, MultiSeedGen, addresses the problem that blocks the detector more than any architectural choice does. Training an object detector needs images of many seeds with a bounding box drawn around each one, and labelling those boxes by hand is slow and error-prone. What is easy to collect is photographs of single seeds. MultiSeedGen turns the cheap input into the expensive output. It segments individual seeds out of source photos using a mix of classical masking and learned matting [21], with an optional Segment Anything backend for harder cases [22], then composites many of those cutouts onto realistic backgrounds. Because the tool places each cutout itself, it knows exactly where every seed sits, so the bounding boxes come for free as a by-product of placement. The result is exported as a detection dataset in YOLO and COCO formats [3], [17], [20].

To make the output useful for training rather than a toy, the compositor applies domain-matched lighting and camera degradation, a move toward closing the gap between synthetic and real images through domain randomisation and augmentation [4], [23], [24]. This is the same cut-and-paste synthesis idea that prior work used to bootstrap detectors, applied to the seed-grading setting. Every run also writes a record of the configuration that produced it, so a dataset can be regenerated and an experiment can be repeated.

Taken together, the two systems form a loop. MultiSeedGen produces the labelled detection data the pipeline's detector is trained on, and the pipeline produces the predictions that can later tell MultiSeedGen which kinds of scenes it is generating too few of. Closing that loop in practice is one of the future enhancements in Section 7.3.

7.1.2 What this validates

The objective set in Chapter 1 was to show that seed-quality grading can be automated on ordinary hardware rather than the specialised imaging rigs used in commercial seed-testing systems. The prototype does this. It runs on commodity machines, takes ordinary photographs as input, and produces per-seed quality labels with measured accuracy on two distinct crops. Together with MultiSeedGen, which makes the training data for new crops cheap to produce, the work shows that automated seed-quality grading is feasible without expensive equipment, and that the data bottleneck which usually stalls such projects can be worked around.

7.2 Lessons learned

The clearest lesson is that the training data mattered more than the model architecture. Most of the early accuracy gains came from collecting more varied and cleaner seed images and from fixing labelling mistakes, not from swapping backbones or tuning the network. A detector trained on clean, repetitive synthetic scenes did well on synthetic test images and poorly on real photographs. Adding variety in lighting, background, and seed arrangement closed much more of

that gap than any change to the model did. This matches the wider finding in agricultural deep learning that data quality and quantity tend to dominate [1], [2].

Splitting the problem into a detector and a separate classifier was the right call. Because detection and classification are independent stages, we could diagnose and improve each one without disturbing the other. When the classifier confused good and bad maize seeds, we could retrain it alone. When the detector missed seeds that overlapped, we could change the detection training data alone. Keeping the two halves separate kept the work tractable and kept each metric meaningful on its own.

The most expensive lesson came from synthetic data and evaluation. It is easy to produce metrics that look excellent and mean nothing, because seeds cut from the same source photos can end up in both the training set and the test set. When that happens the model is partly tested on seeds it has already seen, and the reported accuracy is inflated. We had to build a leakage-safe split that keeps all crops derived from one source image on the same side of the train/test divide. Before that fix the numbers were optimistic; after it they were lower but trustworthy. Synthetic data narrows the labelling bottleneck, but it does not remove the distribution shift between generated and real images, and the only fair way to measure whether it has helped is to test against real labelled photographs rather than against more synthetic data.

7.3 Future enhancements

The prototype proves the concept. Several lines of work would extend it and make it more useful in the field.

The first is supporting more crops. The pipeline handles coffee and maize today, and it was built to extend: a new crop is a new detector class plus a new per-crop classifier. On the data side MultiSeedGen extends the same way, since a new crop is a new set of source seed photos and a configuration recipe rather than a change to the tool. Public seed datasets for crops such as corn could seed this work [26], [27], [28].

The second is a mobile capture app so a farmer can photograph seeds in the field and get a result on the spot. A lightweight detector and classifier, for example a compact YOLO model [16], [17] built with a mobile-friendly framework [29], would let the grading run on the device with no round trip, which matters where connectivity is poor.

The third is narrowing the synthetic-to-real domain gap and validating the detector on genuine field photographs. The current models are trained largely on generated scenes, so the most important test is a held-out set of real, hand-labelled field images. Measuring the detector there, and feeding what we learn back into MultiSeedGen's degradation and randomisation, is the direct way to shrink the gap between how the system performs on synthetic data and how it performs in the field.

The fourth, and the one that ties the others together, is an active-learning loop. Real scans that the system grades with low confidence, or that a user corrects, are exactly the cases the

current models handle worst, and they describe the scenes the generator is underproducing. The loop is to collect those hard real scans, feed their characteristics back into MultiSeedGen to regenerate training data weighted toward the failure modes, retrain the detector and the affected classifier, evaluate the candidate against a held-out set of real labelled scans, and ship it only if it improves. Each turn of the loop targets generation at the system's measured weaknesses instead of producing more of what it already handles well.

Beyond these, taking the prototype to production would mean hardening the surrounding software and operations, but the grading capability itself is in place. The work in this report shows that automated seed-quality grading on commodity hardware is feasible, and that generated training data can carry it most of the way there.

References

[1] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Computers and Electronics in Agriculture*, vol. 147, pp. 70–90, 2018.

[2] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, p. 1419, 2016.

[3] D. Dwibedi, I. Misra, and M. Hebert, "Cut, paste and learn: Surprisingly easy synthesis for instance detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 1301–1310.

[4] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.

[5] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: convolutional block attention module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.

[6] S. Ramírez, "Fastapi framework, high performance, easy to learn, fast to code, ready for production," 2018. [Online]. Available: https://github.com/tiangolo/fastapi

[7] M. Go, *React 18 Design Patterns and Best Practices*. Packt Publishing, 2022.

[8] D. Merkel, "Docker: Lightweight linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.

[9] LemnaTec GmbH, *SeedAIxpert: High-throughput digital seed testing and phenotyping*, LemnaTec product documentation, 2026. [Online]. Available: https://www.lemnatec.com/

[10] PCS Agri, *Track seeds: Digital seed germination management platform*, PCS Agri product portfolio, 2026. [Online]. Available: https://www.pcs-agri.com/track-seeds

[11] Ideas All Day Ltd., *Seedy: Seed identifier*, Apple App Store, 2026. [Online]. Available: https://apps.apple.com/us/app/seed-identifier-seedy/id6749047178

[12] D. Grimm et al., *GerminationPrediction: Machine-learning-based germination detection*, GitHub repository, grimmlab, 2024. [Online]. Available: https://github.com/grimmlab/GerminationPrediction

[13] Multi-View Oil Palm Seed Research Project, *Multi-view convolutional neural networks for oil palm seed quality assessment*, Research project, 2022.

[14] S. Ren, K. He, R. B. Girshick, and J. Sun, "Faster R-CNN: towards real-time object detection with region proposal networks," *CoRR*, vol. abs/1506.01497, 2015. [Online]. Available: http://arxiv.org/abs/1506.01497

[15] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.

[16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[17] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: http://arxiv.org/abs/1512.03385

[19] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: a large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[20] T.-Y. Lin et al., "Microsoft COCO: common objects in context," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.

[21] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U2-Net: going deeper with nested u-structure for salient object detection," *Pattern Recognition*, vol. 106, p. 107 404, 2020.

[22] A. Kirillov et al., "Segment anything," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4015–4026.

[23] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, pp. 1–48, 2019.

[24] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.

[25] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[26] A. Naagar et al., *NAGAR: A large-scale dataset for corn seed quality assessment*, Online dataset, 2025. [Online]. Available: https://naagar.github.io/cornseedsdataset/

[27] Roboflow Universe, *Seed quality detection dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/detecction-ivjfi/detecction

[28] Roboflow Universe, *Maize quality assessment dataset*, Roboflow Universe, 2025. [Online]. Available: https://universe.roboflow.com/grad-project-seed-bank/maize-gslkp-uhnsk

[29] E. Team, *Expo SDK 52: Cross-Platform Native Mobile Development*. O'Reilly Media, 2024.